



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Redes neuronales para imágenes

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

14.1	Introducción	1
14.2	Capas comunes	6
14.2.1	Capas convolucionales	6
14.2.1.1	Convolución en 1d	6
14.2.1.2	Convolución en 2d	10
14.2.1.3	Convolución como producto matriz-vector	13
14.2.1.4	Condiciones de contorno y relleno	14
14.2.1.5	Convolución a saltos	17
14.2.1.6	Múltiples canales de entrada y salida	19
14.2.1.7	Convolución 1x1 (punto a punto)	21
14.2.2	Capas de agrupación	22
14.2.3	Todo junto	24
14.2.4	Capas de normalización	25
14.2.4.1	Normalización batch	26
14.2.4.2	Otros tipos de capa de normalización	28
14.2.4.3	Redes libres de normalización	29
14.3	Arquitecturas comunes para clasificación de imágenes	30

14.3.1	LeNet	31
14.3.2	AlexNet	35
14.3.3	GoogLeNet (Inception)	38
14.3.4	ResNet	39
14.3.5	DenseNet	41
14.3.6	Búsqueda de arquitecturas neuronales	43
14.4	Otras formas de convolución	44
14.4.1	Convolución dilatada	44
14.4.2	Convolución transpuesta	46
14.4.3	Convolución separable en profundidad	48
14.5	CNNs para otras tareas de visión	49
14.5.1	Etiquetado de imágenes	49
14.5.2	Detección de objetos	50
14.5.3	Segmentación de instancias	51
14.5.4	Segmentación semántica	52
14.5.5	Estimación de pose humana	56

14.1. Introducción

► *De datos “desestructurados” a estructurados:*

- Los MLPs del capítulo 13 aprenden a transformar vectores de entrada “desestructurados”, $x \in \mathbb{R}^D$, en salidas
- Ahora extendemos los MLPs al caso en el que la entrada x tiene estructura espacial 2d
- Se pueden aplicar ideas similares en el caso de una estructura temporal 1d, o de una estructura espacio-temporal 3d

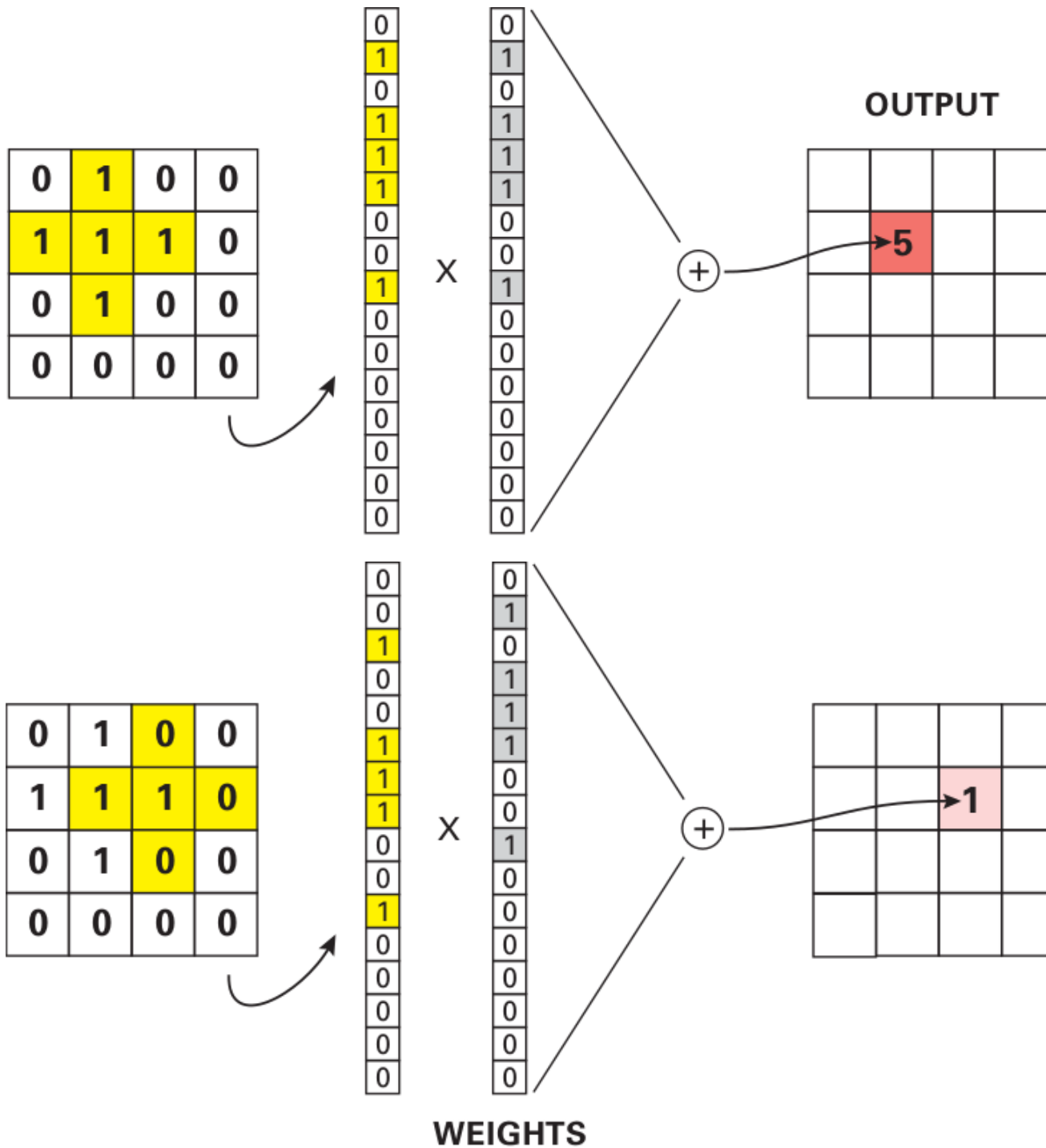
► **Los MLPs no se aplican directamente a imágenes:**

▷ **La operación básica puede verse como un matching:**

- ⇒ Para cada elemento j de cada capa, $z_j = \varphi(w_j^t x)$ viene a ser un matching entre la entrada x y un template aprendido w_j
- ⇒ Si el encaje es bueno ($w_j^t x$ elevado), la activación correspondiente también lo será (con una ReLU), lo cual viene a indicar que el patrón j se halla presente en x

▷ **Número de parámetros prohibitivo:** a causa del elevado número de parámetros requerido, no nos planteamos aprender un W independiente por cada posible tamaño, si las imágenes son de tamaño variable; ni siquiera un único W si son de tamaño fijo

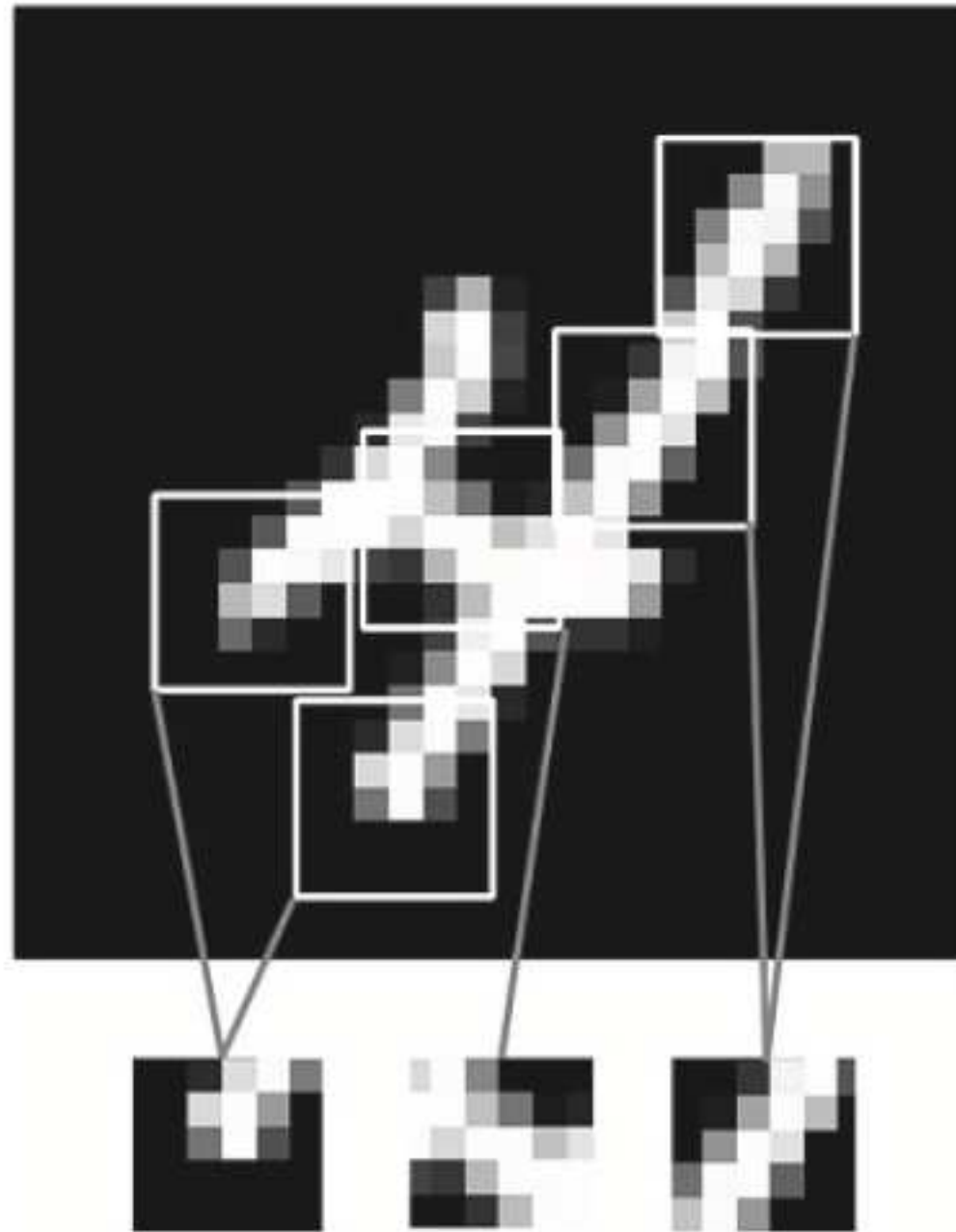
▷ **Falta de invarianza a la traslación:** un patrón localizado en una cierta posición puede no ser localizado en otra distinta ya que los pesos no se comparten entre posiciones distintas



► *Convolutional neural networks (CNNs):*

- ▷ El producto matricial se sustituye por una convolución
- ▷ La imagen se divide en pedazos (*patches*) solapados y cada uno se compara con un conjunto de matrices de pesos pequeñas, o *filtros*, que representan partes de un objeto
- ▷ Se puede ver como una forma de *template matching*
- ▷ Los filtros suelen ser pequeños (3x3 o 5x5) y se aprenden a partir de los datos, por lo que vienen a ser una extracción de características invariante a traslaciones integrada en el modelo

▷ *Ejemplo:* dígito clasificado mediante patrones locales



14.2. Capas comunes

14.2.1. Capas convolucionales

14.2.1.1. Convolución en 1d

► *Convolución continua:*

► Convolución de dos funciones, $f, g : \mathbb{R}^D \rightarrow \mathbb{R}$:

$$[f \circledast g](z) = \int_{\mathbb{R}^D} f(u)g(z - u)du \quad (1)$$

► Aplicamos el *filtro* o *kernel* f a g en z , esto es, hallamos una “media” de g según f centrado en z

► *Convolución discreta:*

- ▷ Reemplazamos las funciones por vectores de longitud finita, esto es, funciones definidas sobre un conjunto finito de puntos
- ▷ f se evalúa en $-L, -L + 1, \dots, 0, 1, \dots, L$, dando lugar al vector de pesos de $w_{-L} = f(-L)$ a $w_L = f(L)$
- ▷ g se evalúa en los puntos $\{-N, \dots, N\}$, dando lugar al vector de características de $x_{-N} = g(-N)$ a $x_N = g(N)$
- ▷ Salvando condiciones de contorno (efectos arista), (1) es:

$$[\mathbf{w} \otimes \mathbf{x}](i) = w_{-L}x_{i+L} + \dots + w_{-1}x_{i+1} + w_0x_i + w_1x_{i-1} + \dots + w_Lx_{i-L} \quad (2)$$

- ▷ El vector de pesos $\mathbf{w}$ se invierte (ya que sus índices se invierten) y se desplaza sobre $\mathbf{x}$, sumando variables locales en cada punto

▷ *Ejemplo:* convolución discreta de $\mathbf{x} = [1, 2, 3, 4]$ con $\mathbf{w} = [5, 6, 7]$, para dar lugar a $\mathbf{z} = [5, 16, 34, 52, 45, 28]$:

—	—	1	2	3	4	—	—	
7	6	5	—	—	—	—	—	$z_0 = x_0 w_0 = 5$
—	7	6	5	—	—	—	—	$z_1 = x_0 w_1 + x_1 w_0 = 16$
—	—	7	6	5	—	—	—	$z_2 = x_0 w_2 + x_1 w_1 + x_2 w_0 = 34$
—	—	—	7	6	5	—	—	$z_3 = x_1 w_2 + x_2 w_1 + x_3 w_0 = 52$
—	—	—	—	7	6	5	—	$z_4 = x_2 w_2 + x_3 w_1 = 45$
—	—	—	—	—	7	6	5	$z_5 = x_3 w_2 = 28$

$$\mapsto i=0: [\mathbf{w} \circledast \mathbf{x}](0) = w_{-1}x_{0+1} + w_0x_0 + w_1x_{0-1} = x_0w_0$$

$$\mapsto i=1: [\mathbf{w} \circledast \mathbf{x}](1) = w_{-1}x_{1+1} + w_0x_1 + w_1x_{1-1} = x_0w_1 + x_1w_0$$

$$\mapsto i=2: [\mathbf{w} \circledast \mathbf{x}](2) = w_{-1}x_{2+1} + w_0x_2 + w_1x_{2-1} = x_0w_2 + x_1w_1 + x_2w_0$$

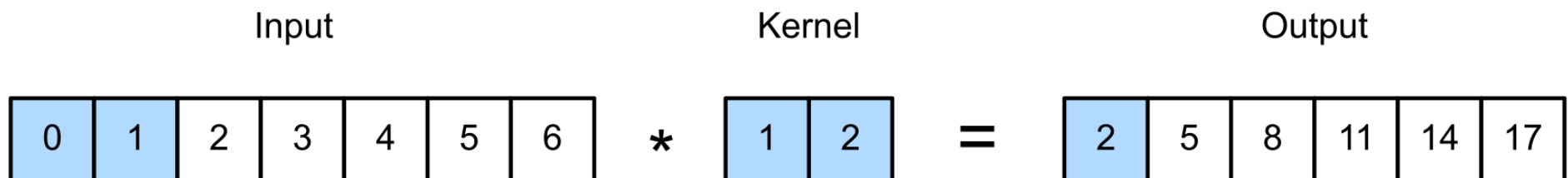
► Correlación cruzada:

- Convolución sin invertir w :

$$[\mathbf{w} * \mathbf{x}](i) = w_{-L}x_{i-L} + \dots + w_{-1}x_{i-1} + w_0x_i + w_1x_{i+1} + \dots + w_Lx_{i+L} \quad (3)$$

- En aprendizaje automático, convolución suele referirse a correlación cruzada; de hecho, coinciden si w es simétrico
- Si w se evalúa en $\{0, 1, \dots, L - 1\}$ y x en $\{0, 1, \dots, N - 1\}$ para evitar índices negativos, entonces:

$$[\mathbf{w} \circledast \mathbf{x}](i) = \sum_{u=0}^{L-1} w_u x_{i+u} \quad (4)$$



14.2.1.2. Convolució en 2d

► *Definició:*

► En 2d, (4) es:

$$[\mathbf{W} \circledast \mathbf{X}](i, j) = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v} x_{i+u, j+v} \quad (5)$$

donde $\mathbf{W}$ es un filtro 2d de tamaño $H \times W$

▷ *Ejemplo:* convolución de $\mathbf{X} 3 \times 3$ con $\mathbf{W} 2 \times 2$ que da $\mathbf{Y} 2 \times 2$

$$\mathbf{Y} = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} \circledast \begin{bmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{bmatrix} \quad (6)$$

$$= \begin{bmatrix} (w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5) & (w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6) \\ (w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8) & (w_1x_5 + w_2x_6 + w_3x_8 + w_4x_9) \end{bmatrix} \quad (7)$$

0	1	2
3	4	5
6	7	8

*

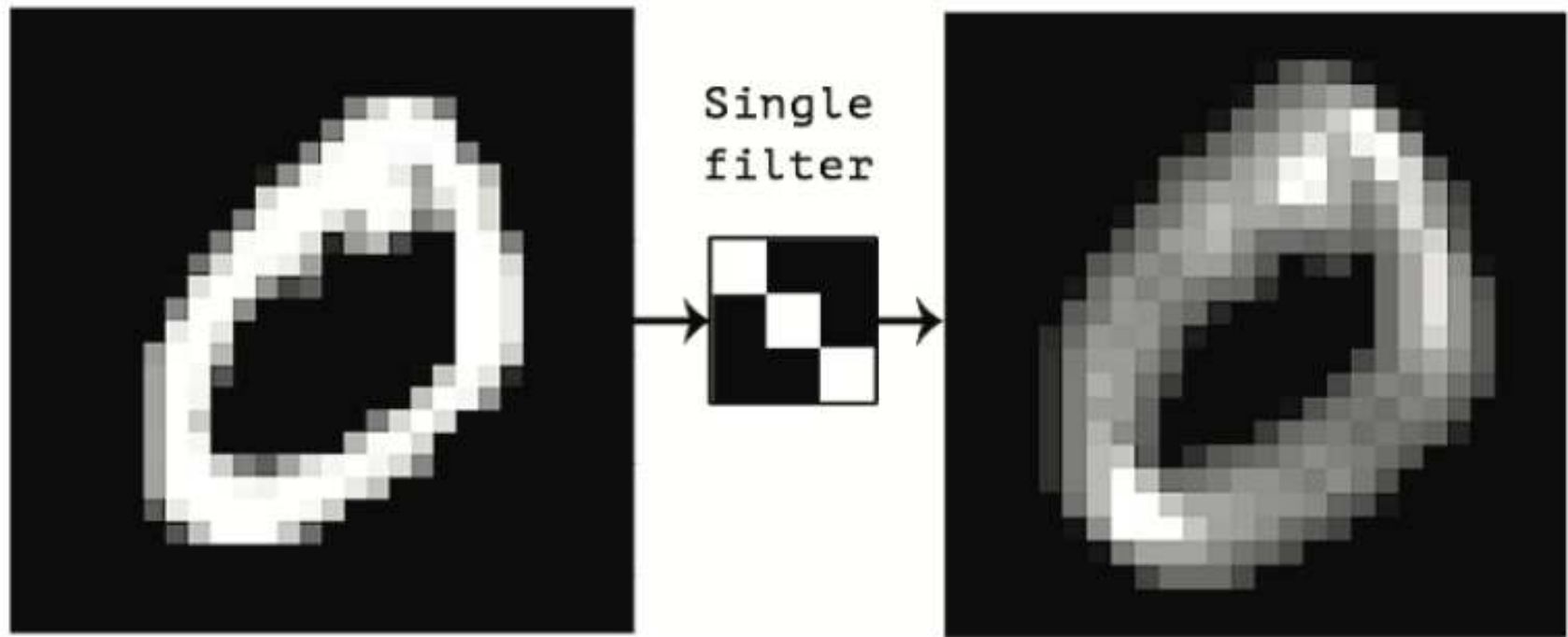
0	1
2	3

=

19	25
37	43

► Interpretación:

- **Template matching:** la salida en (i, j) será grande si el correspondiente trozo de imagen centrado en (i, j) es similar a W
- **Heat map** caliente en las regiones que contengan aristas que encajen con la arista orientada dada por plantilla W



- **Feature map:** llamamos así a $Y = W \circledast X$ ya que la convolución puede verse como una forma de **feature detection**

14.2.1.3. Convolución como producto matriz-vector

- ▶ Dado que la convolución es un operador lineal, puede representarse mediante multiplicación matricial
- ▶ *Ejemplo, cont. (7)*: obtenemos una $\mathbf{Y}$ aplanada, $\mathbf{y}$, multiplicando una matriz (Toeplitz) derivada de $\mathbf{W}$, $\mathbf{C}$, por una $\mathbf{X}$ aplanada, $\mathbf{x}$

$$\mathbf{y} = \mathbf{C}\mathbf{x} = \left(\begin{array}{ccc|ccc|ccc} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{array} \right) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix} \quad (8)$$

$$= \begin{pmatrix} w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5 \\ w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6 \\ w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8 \\ w_1x_5 + w_2x_6 + w_3x_8 + w_4x_9 \end{pmatrix} \quad (9)$$

- ▶ Las CNNs son MLPs con matrices de pesos de estructura dispersa especial y elementos ligados en diferentes posiciones

14.2.1.4. Condiciones de contorno y relleno

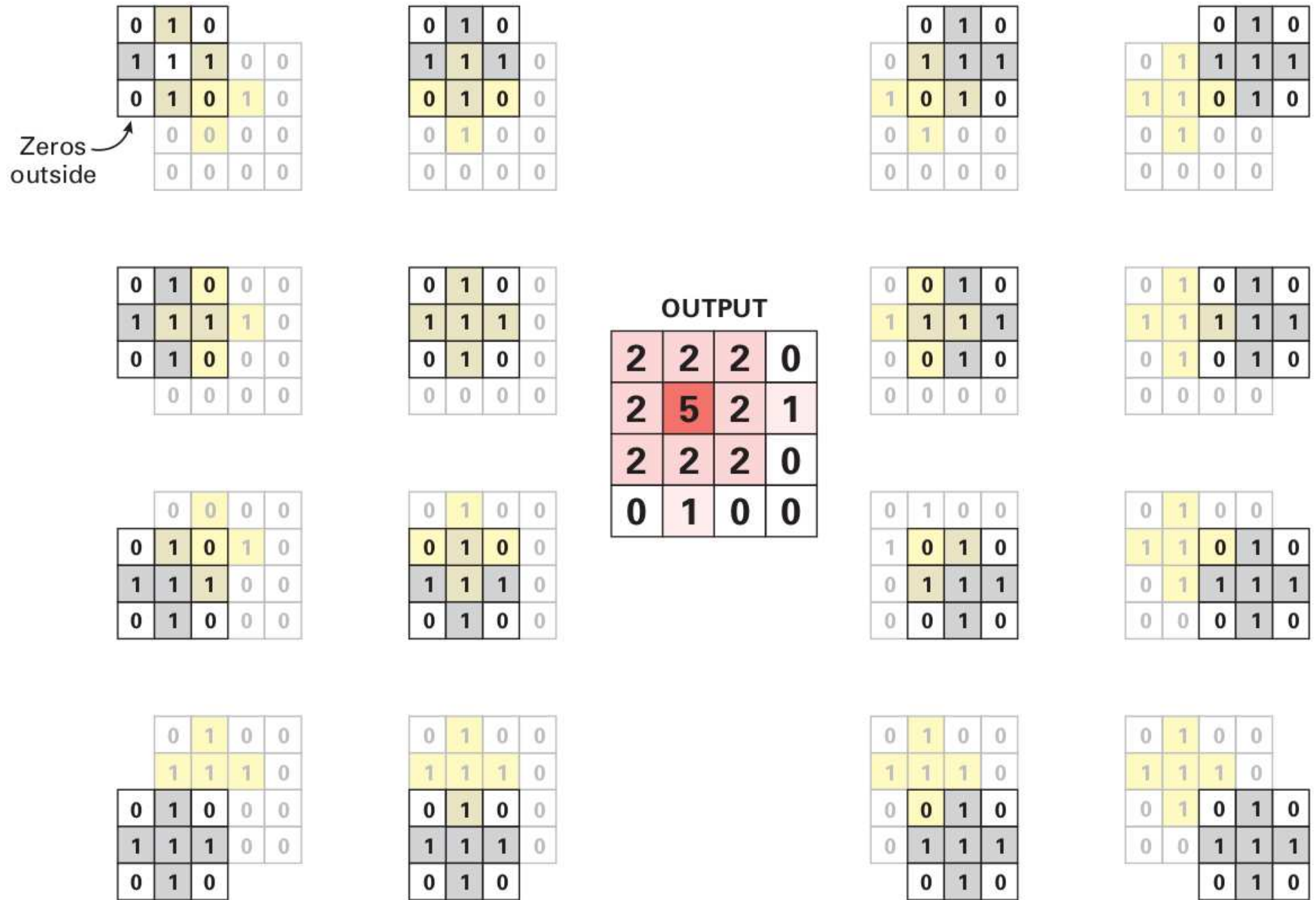
- **Valid convolution:** convolución de un filtro $f_h \times f_w$ con una imagen $x_h \times x_w$ que produce una salida

$$(x_h - f_h + 1) \times (x_w - f_w + 1) \quad (10)$$

- **Same convolution:** añade **relleno cero (zero-padding)** (bordes a cero) para que la salida sea del mismo tamaño que la entrada
- **Caso general:** con entrada $x_h \times x_w$, kernel $f_h \times f_w$, y padding p_h y p_w , el tamaño de la salida es

$$(x_h + 2p_h - f_h + 1) \times (x_w + 2p_w - f_w + 1) \quad (11)$$

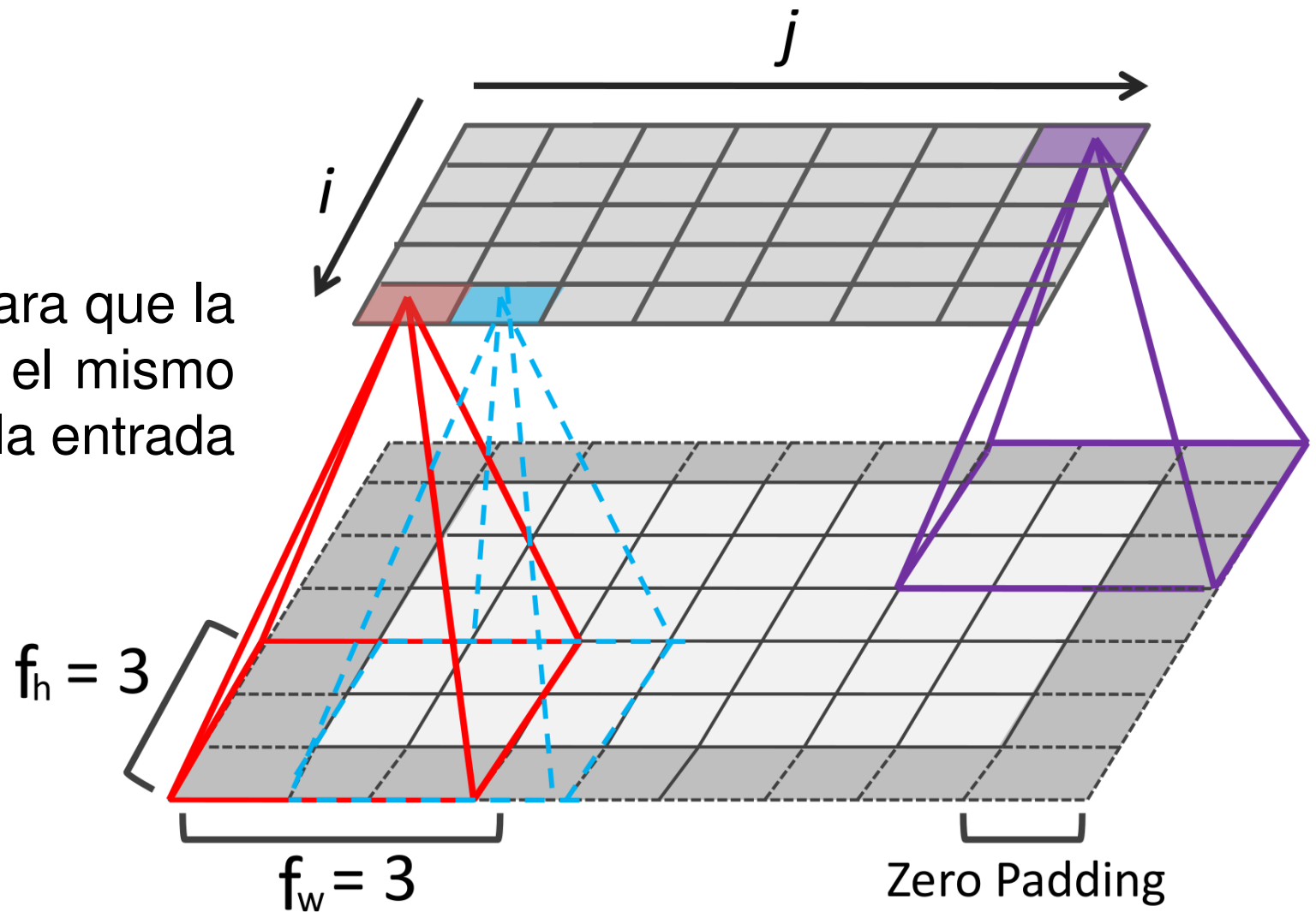
► *Ejemplo:* same convolution con zero-padding



► Ejemplo: same convolution con zero-padding

$$p = 1, f = 3, x_h = 5, x_w = 7 \rightarrow (5 + 2 - 3 + 1) \times (7 + 2 - 3 + 1) = 5 \times 7$$

$2p = f - 1$ para que la salida tenga el mismo tamaño que la entrada



14.2.1.5. Convolución a saltos

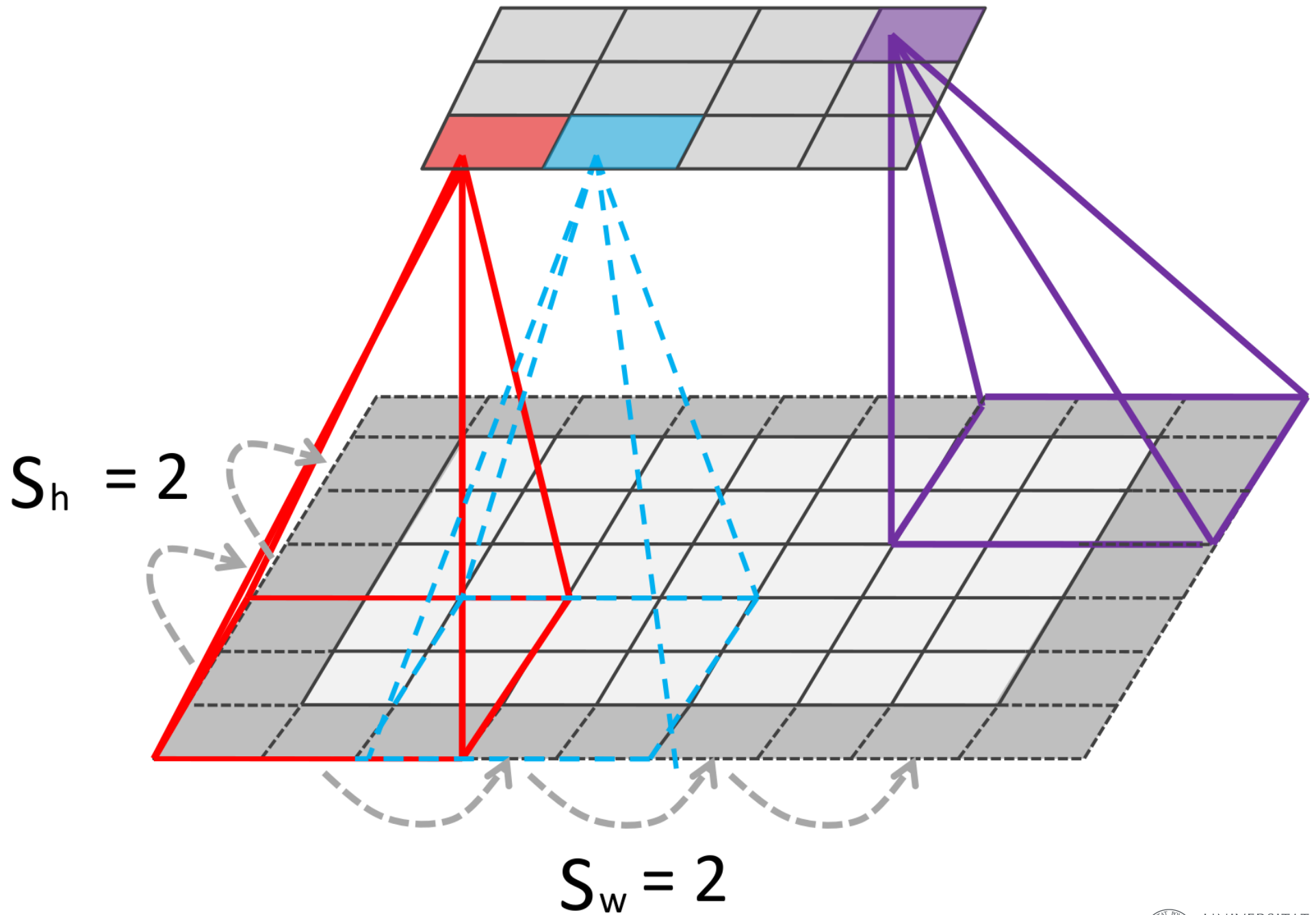
- ▶ Dado que cada píxel de salida es una combinación ponderada de entradas en su **campo receptivo**, los píxeles de salida vecinos tendrán valores similares ya que sus entradas se solapan
- ▶ **Strided convolution:** elimina redundancias y acelera cálculos saltándose cada s -ésima entrada
- ▶ **Tamaño de la salida con saltos:** con entrada $x_h \times x_w$, kernel $f_h \times f_w$, padding p_h y p_w , y strides s_h y s_w , la salida es

$$\left\lfloor \frac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{x_w + 2p_w - f_w + s_w}{s_w} \right\rfloor \quad (12)$$

- ▶ Ejemplo: $p = 1$, $f = 3$, $x_h = 5$, $x_w = 7$ y $s = 2$

$$\left\lfloor \frac{5 + 2 - 3 + 2}{2} \right\rfloor \times \left\lfloor \frac{7 + 2 - 3 + 2}{2} \right\rfloor = 3 \times 4 \quad (13)$$

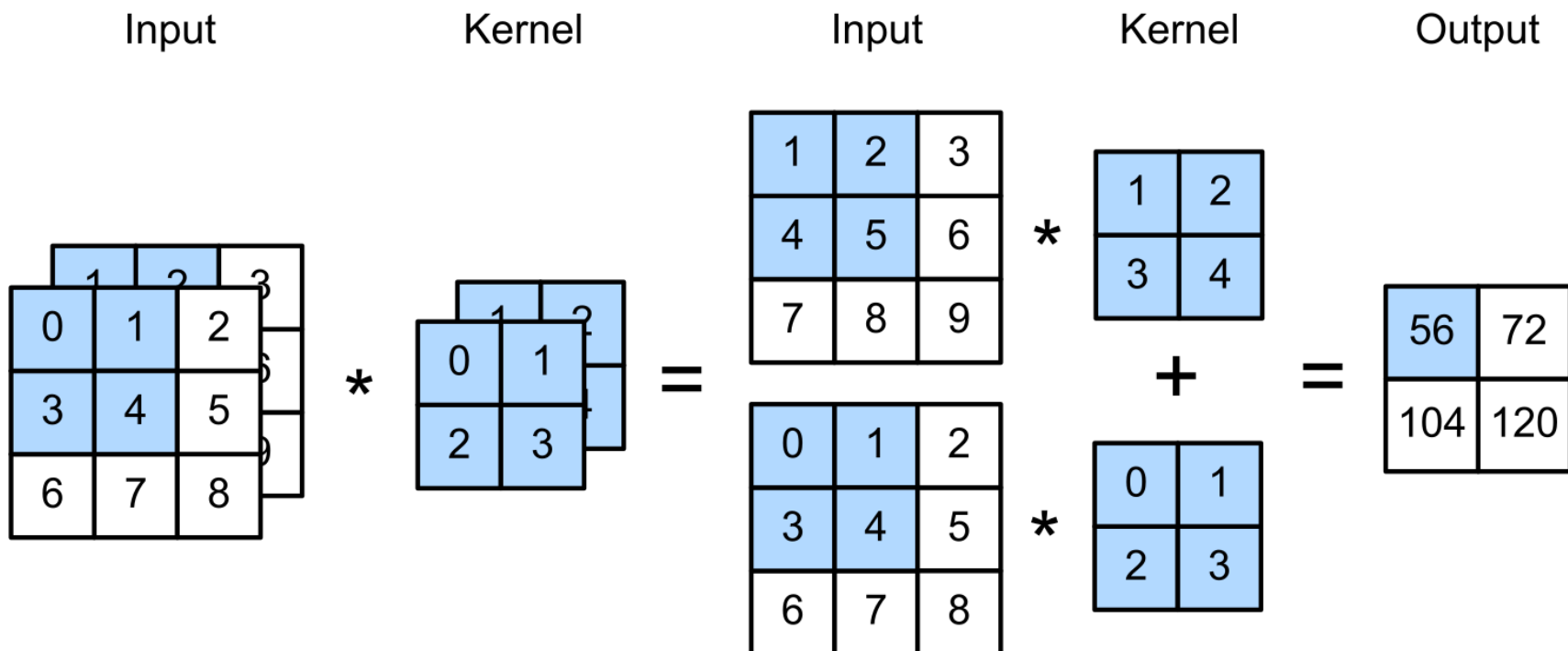
► Ejemplo (cont.): $p = 1, f = 3, x_h = 5, x_w = 7$ y $s = 2 \rightarrow 3 \times 4$



14.2.1.6. Múltiples canales de entrada y salida

- **Múltiples canales de entrada:** con un kernel por cada canal de entrada, W es una matriz de pesos 3d o **tensor** y la salida se obtiene convolucionando el canal c de la entrada con el kernel $W_{::,c}$ y sumando para todos los canales (con sesgo b y salto s):

$$z_{i,j} = b + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u,sj+v,c} w_{u,v,c} \quad (14)$$

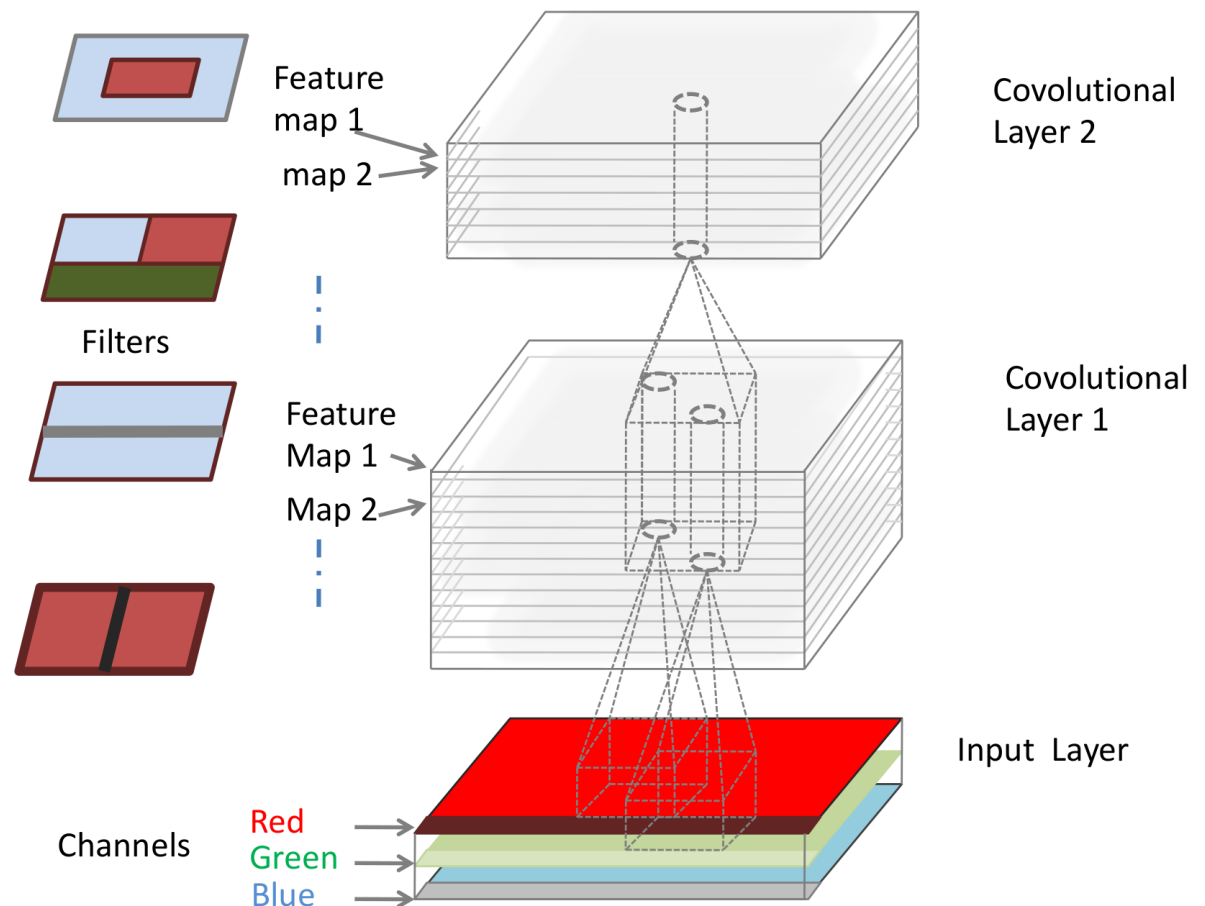


- **Múltiples canales de salida:** con un kernel para cada *tipo de característica* d a detectar en cada canal de entrada c , W es una matriz de pesos 4d con un filtro $W_{:, :, c, d}$ por cada d y c :

$$z_{i,j,d} = b_d + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u,sj+v,c} w_{u,v,c,d} \quad (15)$$

Hipercolumna

Cada columna cilíndrica vertical denota un conjunto de características de salida $z_{i,j,1:D}$

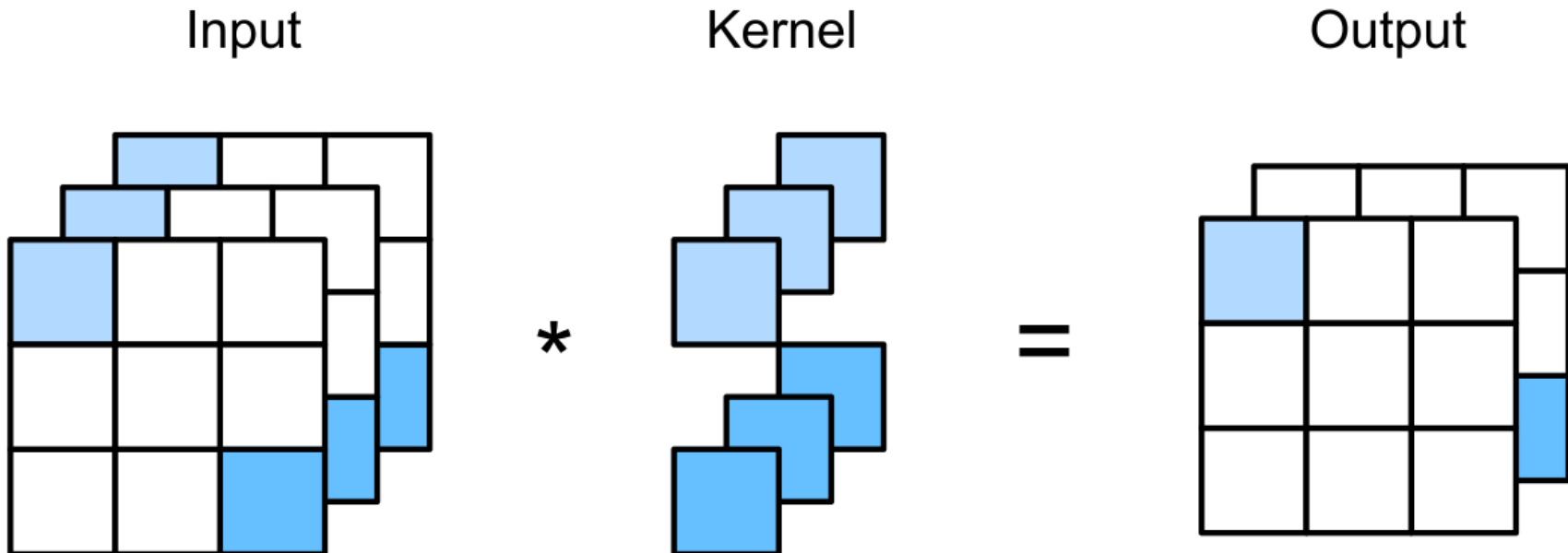


14.2.1.7. Convolución 1x1 (punto a punto)

- **Convolución 1x1 (punto a punto):** combinación ponderada de características en una posición dada, no en varias, por lo que pasamos de C a D canales sin alterar las dimensiones espaciales

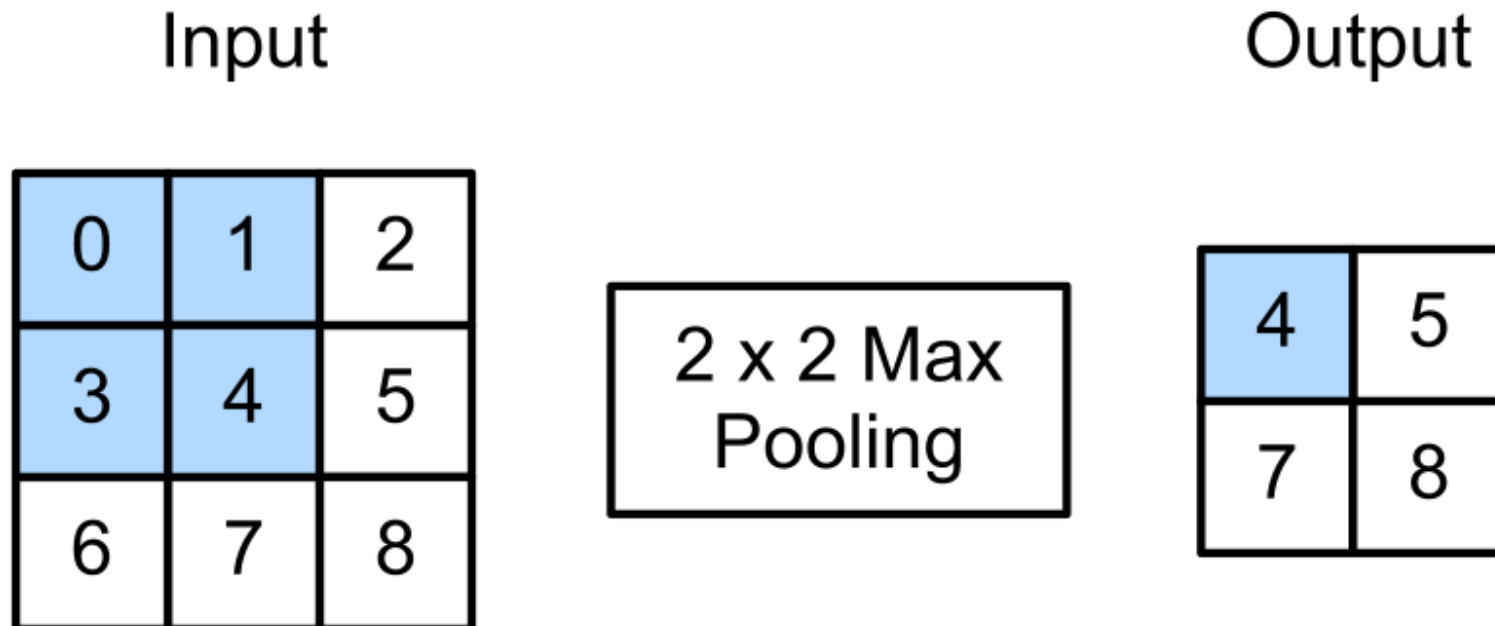
$$z_{i,j,d} = b_d + \sum_{c=0}^{C-1} x_{i,j,c} w_{0,0,c,d} \quad (16)$$

- Ejemplo: de 3 a 2 canales por convolución con filtro $1 \times 1 \times 3 \times 2$



14.2.2. Capas de agrupación

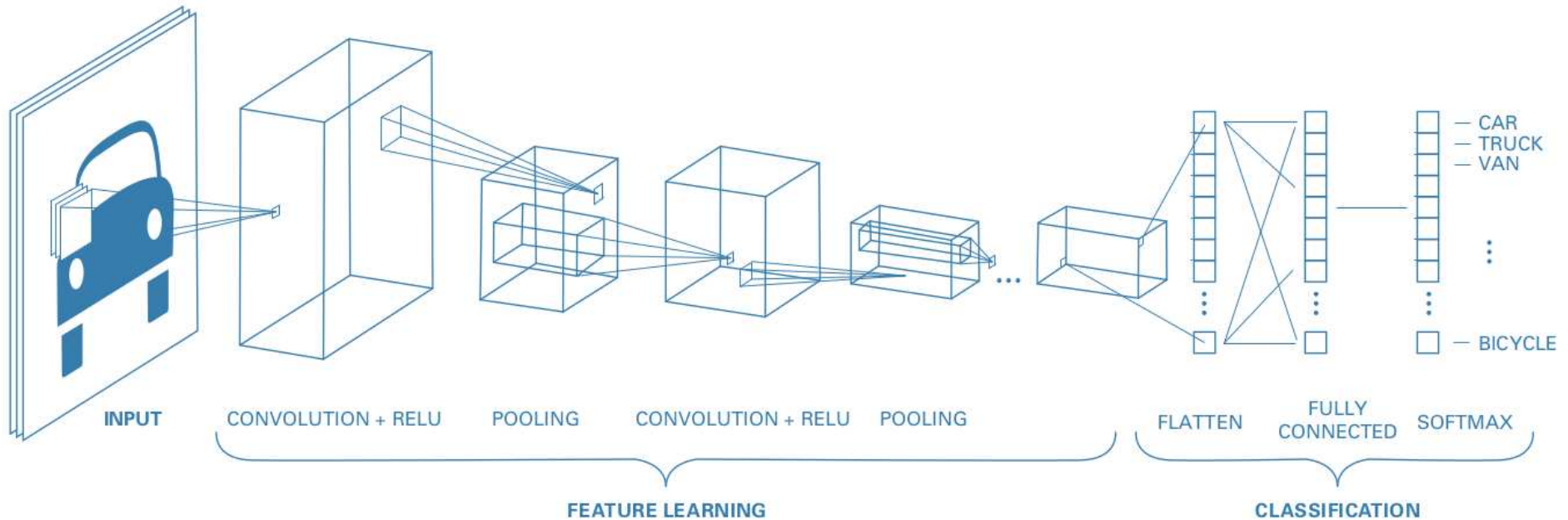
- **Equivarianza de la convolución:** preserva información sobre la posición de las características de entrada (con menor resolución)
- **Invarianza frente a la posición:** si solo se desea saber si un objeto de interés (p.e. una cara) se halla en alguna posición de la imagen, conviene hallar características invariantes a la posición
- **Max pooling:** invarianza por maximización sobre las entradas



- ▶ **Average pooling:** alternativa a max pooling que sustituye máximo por media
- ▶ **Global average pooling:** la media se halla sobre todas las posiciones del mapa de características
 - ▷ Convierte una mapa de características $H \times W \times D$ en un mapa de características $1 \times 1 \times D$ que se pasa como vector D -dimensional a una capa totalmente conectada para transformarlo en un vector C -dimensional de entrada a una salida softmax
 - ▷ Aplicable a la clasificación de imágenes de tamaño variable pues el mapa de características final se convierte en un vector D -dimensional a transformar en una distribución sobre las clases

14.2.3. Todo junto

- **Arquitectura usual de las CNNs:** alternan capas convolucionales con max pooling, más una capa de clasificación lineal al final.



- **Origen:** *Neocognitron* de Fukushima (1975) y *LeNet* de Yann LeCun (1998).

14.2.4. Capas de normalización

- **Objetivo:** añadir capas extra para evitar el problema de los gradientes que desaparecen o explotan en redes profundas

14.2.4.1. Normalización batch

- ▶ Normaliza la distribución de las activaciones de una capa para que tenga media nula y varianza unitaria a nivel de minibatch
- ▶ El vector de activación z_n para la muestra n se reemplaza por

$$\tilde{z}_n = \gamma \odot \hat{z}_n + \beta \quad (17)$$

donde γ y β son parámetros de aprendizaje, mientras que $\hat{z}_n$ es el vector de activación estandarizado en el batch $\mathcal{B}$

$$\hat{z}_n = \frac{z_n - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad (18)$$

con

$$\mu_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{z \in \mathcal{B}} z \quad \text{y} \quad \sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{z \in \mathcal{B}} (z - \mu_{\mathcal{B}})^2 \quad (19)$$

y siendo $\epsilon > 0$ una constante pequeña

- ▶ Dado que la normalización batch es diferenciable, permite retropropagar gradientes a la entrada de la capa y parámetros γ y β
- ▶ **Internal covariate shift:** a diferencia de la media y varianza de la capa de entrada, que se calculan una vez (para datos estáticos), las medias y varianzas de otras capas deben recalcularse en cada minibatch ya que los parámetros cambian durante el aprendizaje
- ▶ **Medias y varianzas en inferencia:** no se calculan a partir del minibatch (unitario) de test, sino que se estiman a partir de todos los datos, tras finalizar el aprendizaje, y se “congelan” para inferencia
- ▶ **Fused batchnorm:** combina una capa batchnorm congelada con su capa previa por eficiencia; si la previa calcula $\mathbf{XW} + \mathbf{b}$, la batchnorm es $\gamma \odot (\mathbf{XW} + \mathbf{b} - \boldsymbol{\mu}) / \sigma + \beta$ o, en combinación, $\mathbf{XW}' + \mathbf{b}'$ con $\mathbf{W}' = \gamma \odot \mathbf{W} / \sigma$ y $\mathbf{b}' = \gamma \odot (\mathbf{b} - \boldsymbol{\mu}) / \sigma + \beta$
- ▶ **Batch renormalization:** variante para mejorar el comportamiento de batchnorm en entrenamiento con minibatches pequeños

14.2.4.2. Otros tipos de capa de normalización

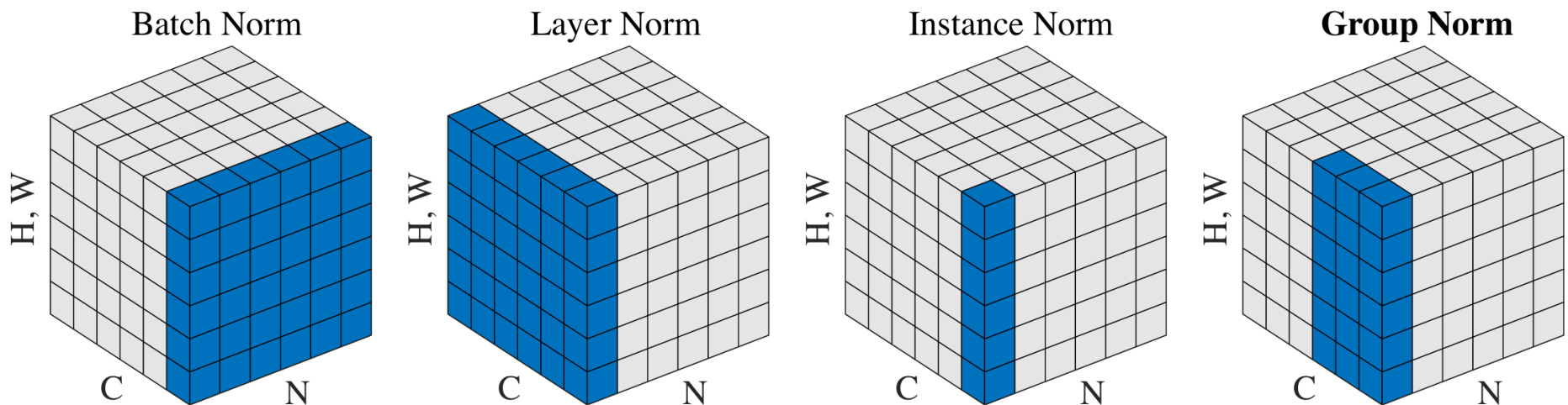
- **Estandarización en dimensiones distintas al batch:** z_i definida en un subconjunto de valores del tensor en su dimensión i , $\mathcal{S}_i$:

$$\tilde{z}_i = \gamma_c \hat{z}_i + \beta_c \quad \text{con} \quad \hat{z}_i = \frac{z_i - \mu_i}{\sigma_i} \quad (20)$$

donde c es el canal correspondiente al índice i ,

$$\mu_i = \frac{1}{|\mathcal{S}_i|} \sum_{k \in \mathcal{S}_i} z_k \quad \text{y} \quad \sigma_i = \sqrt{\frac{1}{|\mathcal{S}_i|} \sum_{k \in \mathcal{S}_i} (z_k - \mu_i)^2 + \epsilon} \quad (21)$$

- **Batch/layer/instance/group norm:** posiciones agrupadas en azul



14.2.4.3. Redes libres de normalización

- ▶ Propuesta reciente para entrenar redes residuales profundas sin capas de normalización con recortado de gradientes adaptativo

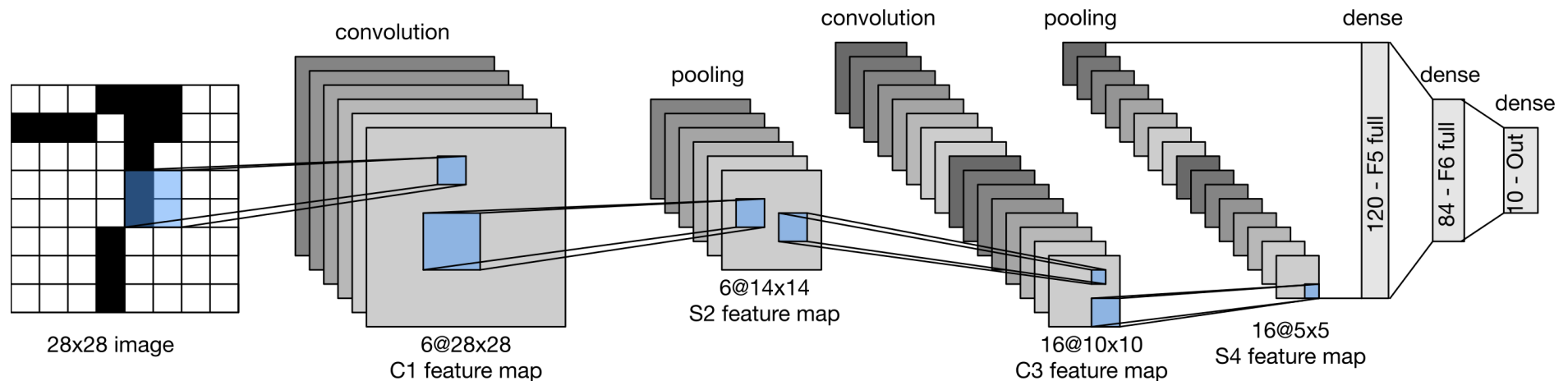
14.3. Arquitecturas comunes para clasificación de imágenes

- ▶ **Clasificación de imágenes:** estimamos $f : \mathbb{R}^{H \times W \times K} \rightarrow \{0, 1\}^C$ donde K es el número de canales y C el número de clases
- ▶ **PyTorch Image Models (timm):** colección de modelos, capas, utilidades, optimizadores, planificadores, cargadores/aumentadores de datos y scripts de entrenamiento/validación para reproducir resultados SOTA en ImageNet

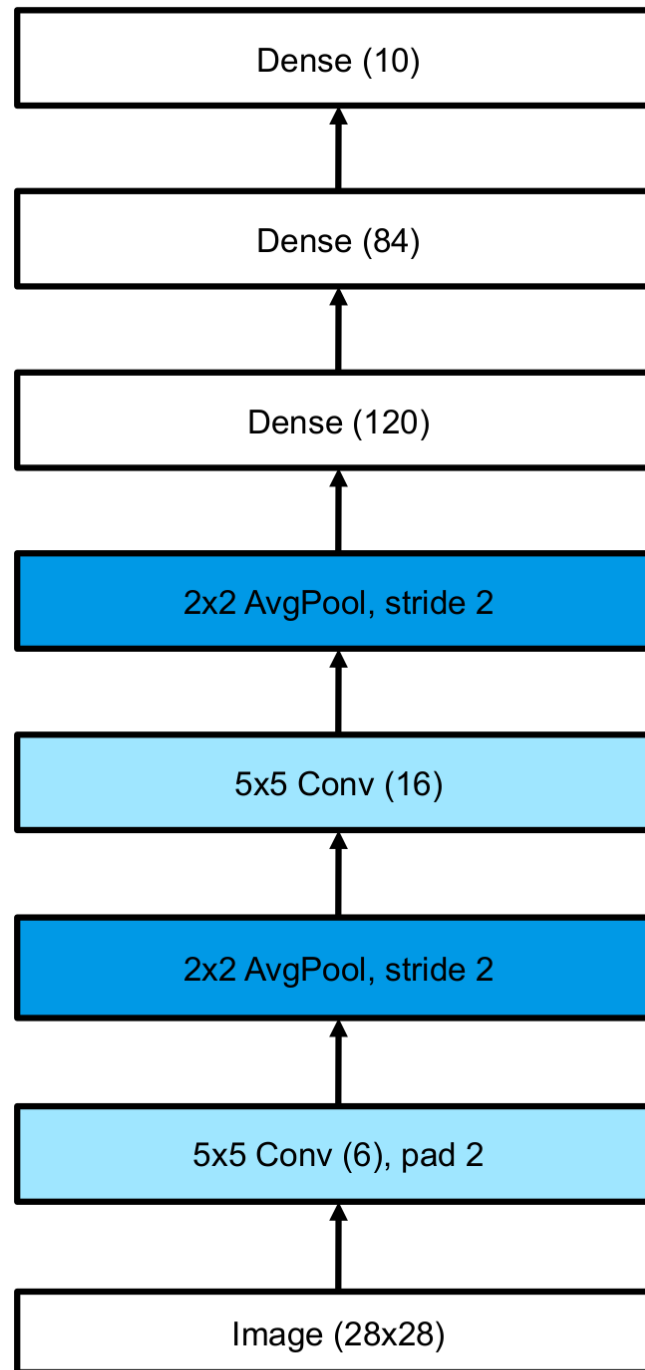
<https://github.com/rwightman/pytorch-image-models>

14.3.1. LeNet

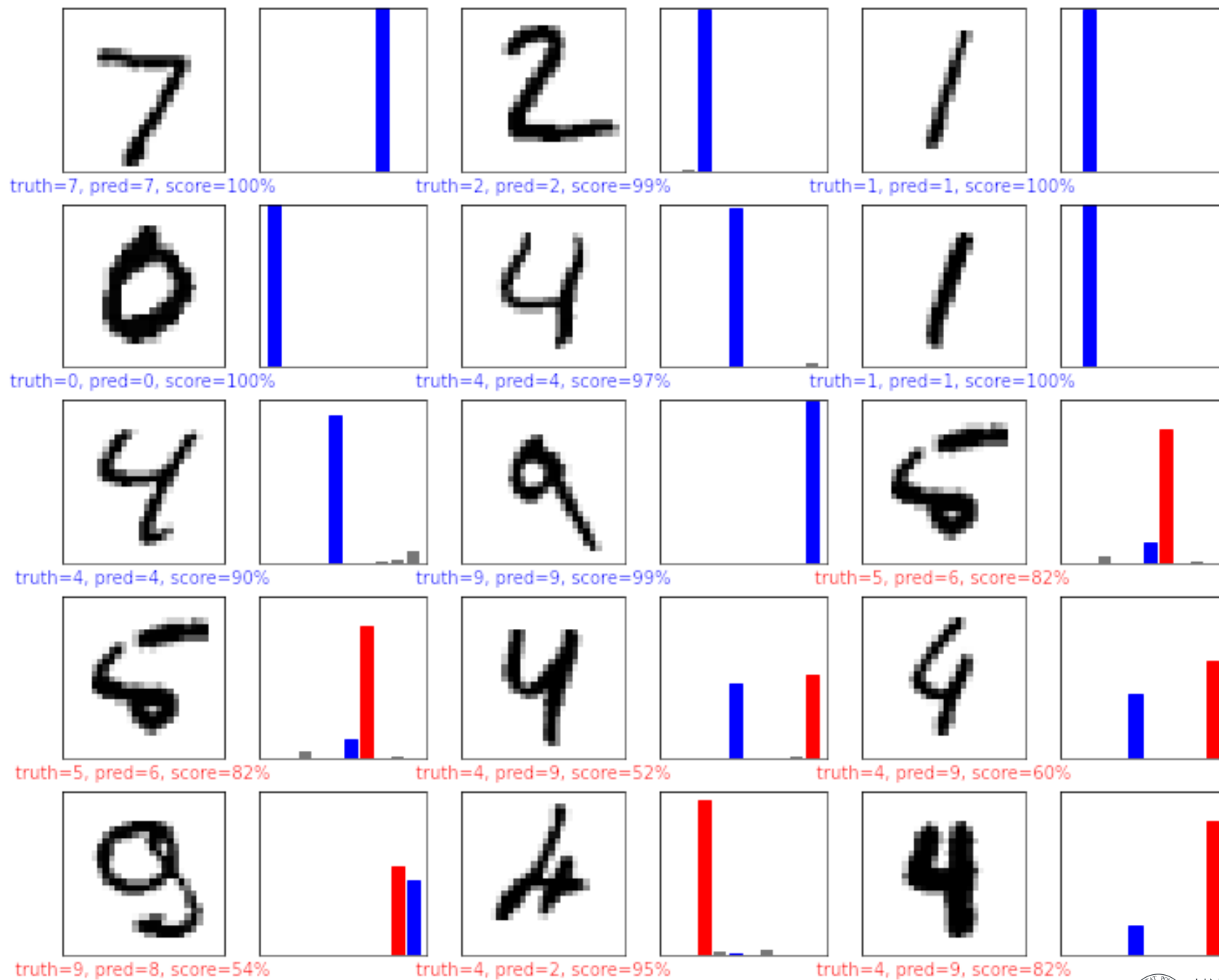
► *LeNet (1998)*: CNN de Yann LeCun para MNIST



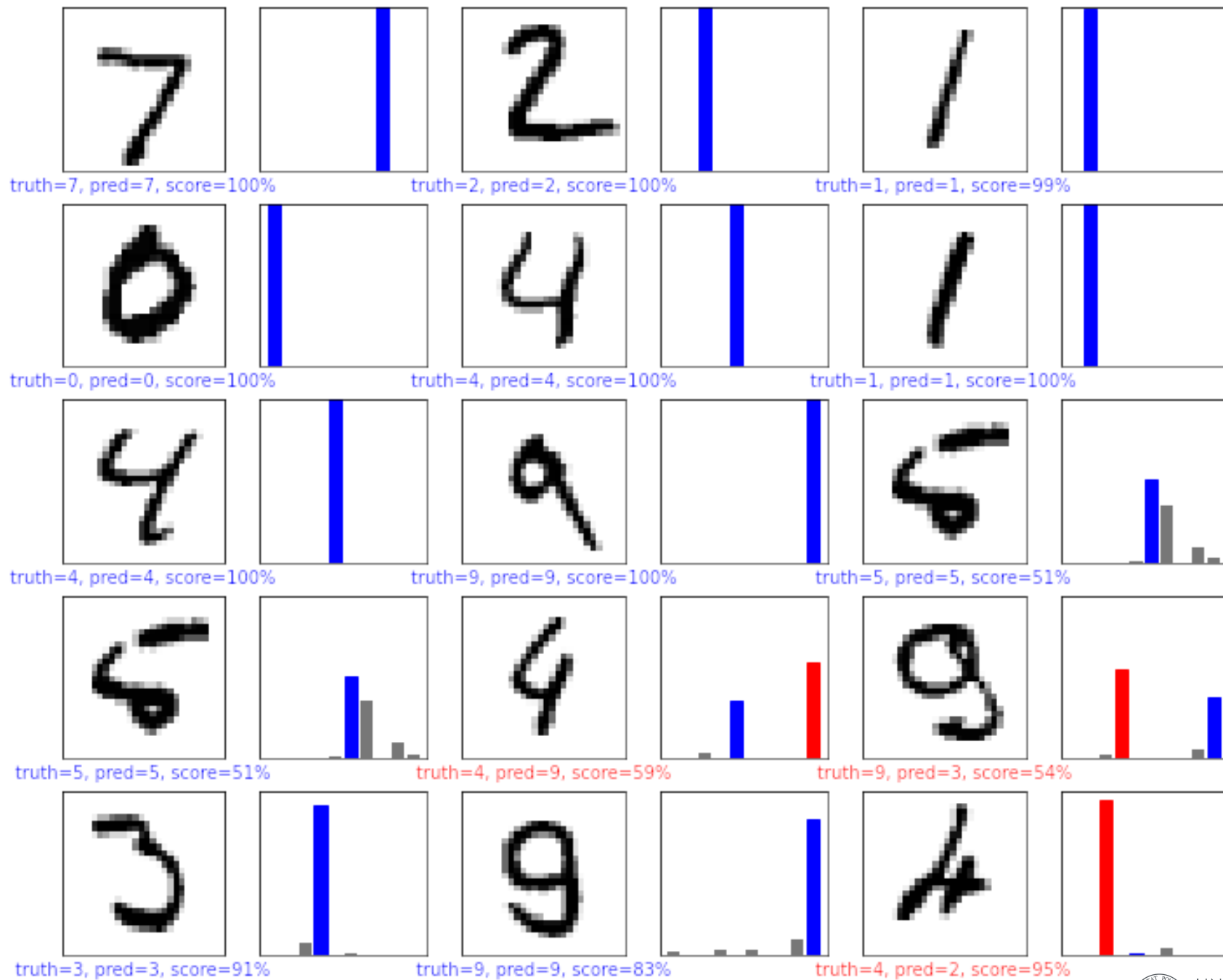
- **C1**: convolución con 6 kernels 5×5 y padding 2
- **S2**: submuestreo con 2×2 avg pooling y stride 2
- **C3**: convolución con 16 kernels 5×5 sin padding
- **S4**: submuestreo con 2×2 avg pooling y stride 2
- **C5/F5**: convolución con 120 kernels 5×5
- **F6**: capa completamente conectada con F5



► Predicciones tras una época de entrenamiento: precisión 98.8 %

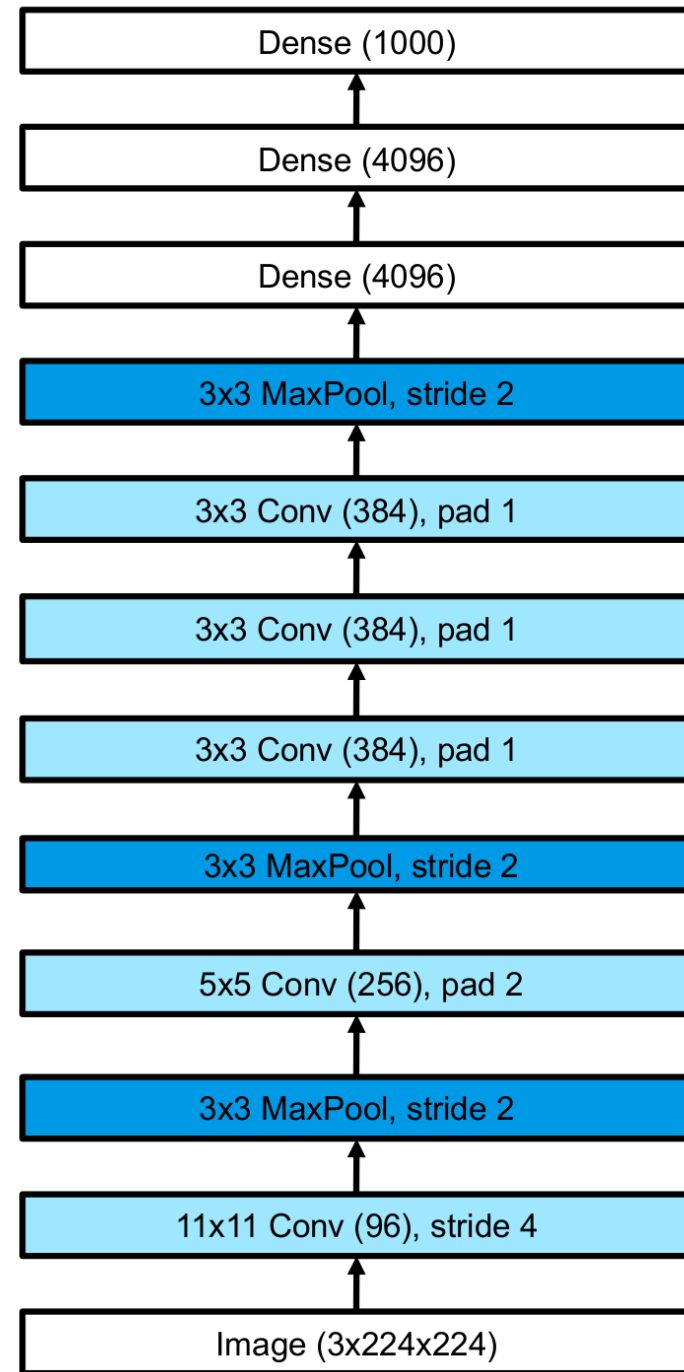
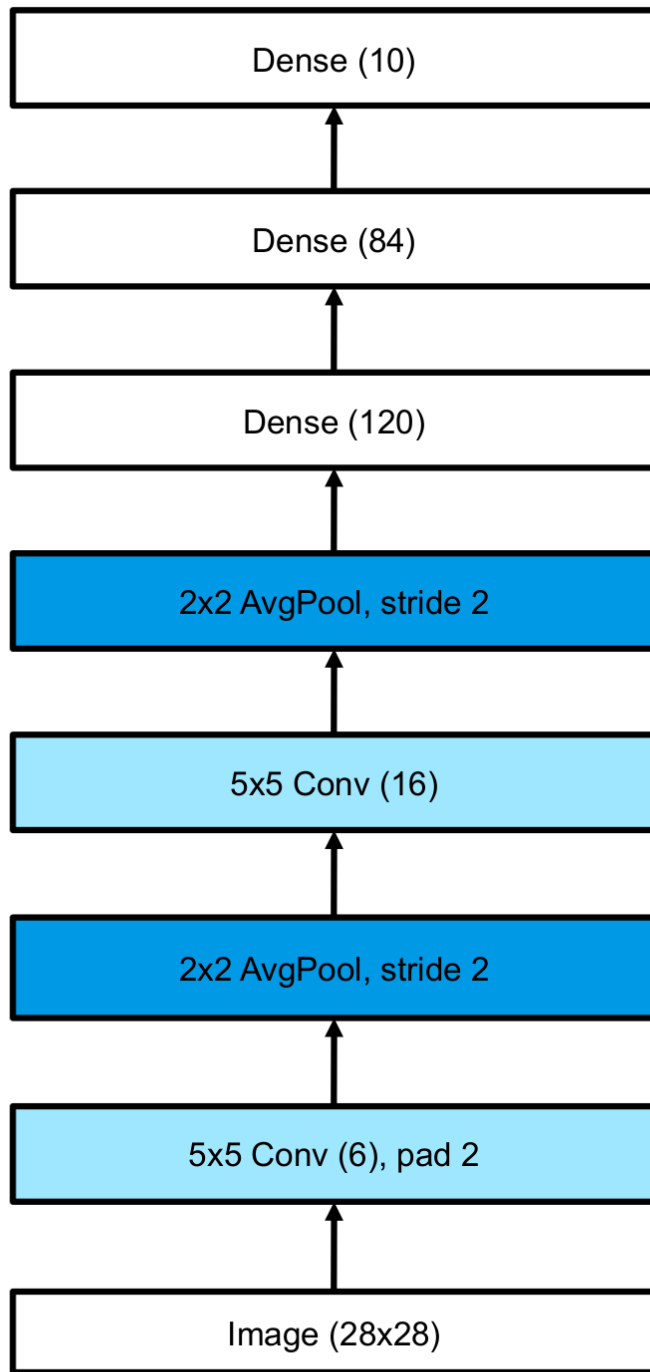


► Predicciones tras dos épocas de entrenamiento:



14.3.2. AlexNet

- ▶ *AlexNet (2012)*: CNN de Alex Krizhevsky para ImageNet
- ▶ Redujo el error del 26 % al 15 % (en top-5 para 1000 clases)
- ▶ Comparación con LeNet:
 - ▷ Más profunda: 8 capas de parámetros ajustables en lugar de 5
 - ▷ Función de activación: ReLU en lugar de tanh
 - ▷ Regularización: dropout en lugar de weight decay
 - ▷ Apila varias capas convolucionales sin alternarlas con pooling





mite

container ship

motor scooter

leopard

	mite		container ship		motor scooter		leopard
	black widow		lifeboat		go-kart		jaguar
	cockroach		amphibian		moped		cheetah
	tick		fireboat		bumper car		snow leopard
	starfish		drilling platform		golfcart		Egyptian cat

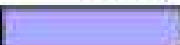
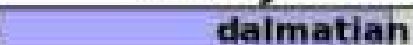


grille

mushroom

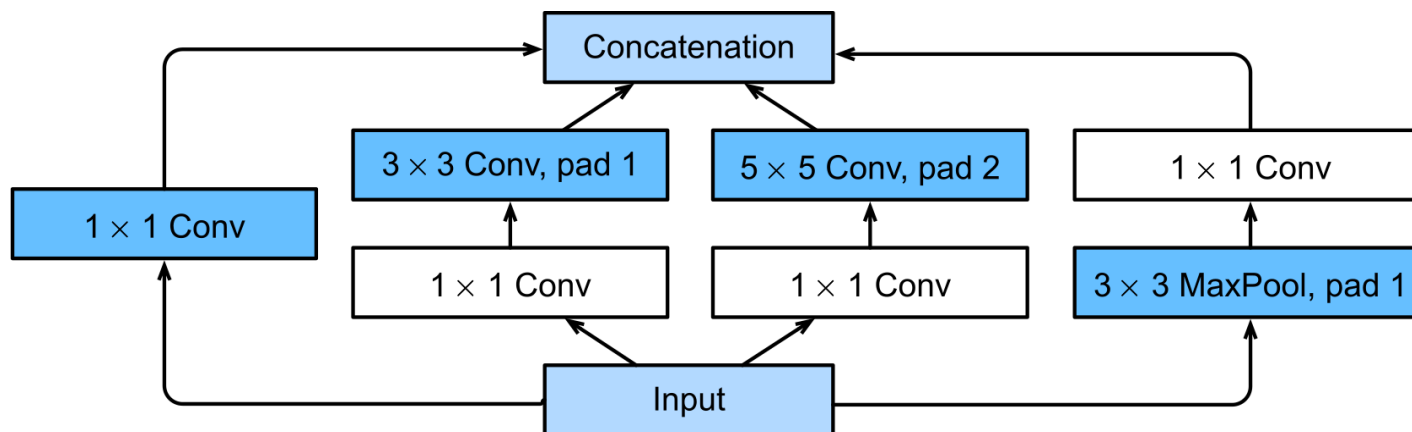
cherry

Madagascar cat

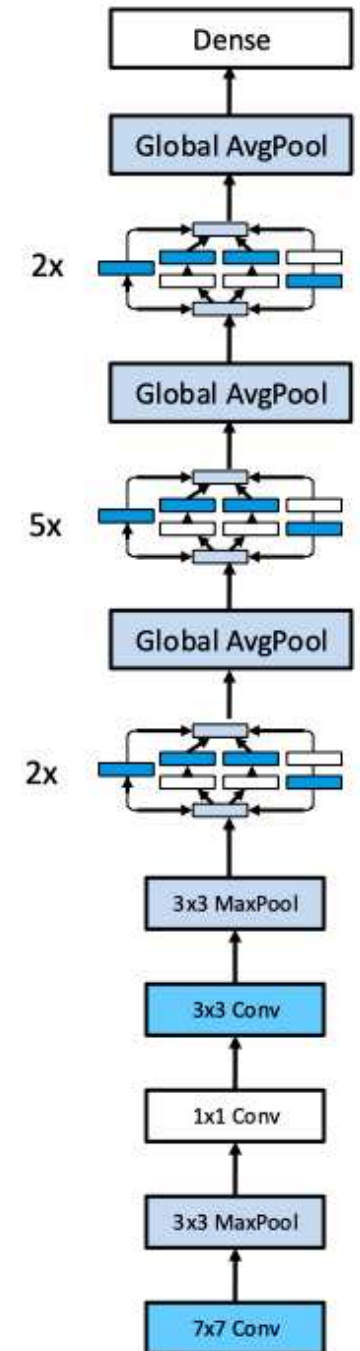
	convertible		agaric		dalmatian		squirrel monkey
	grille		mushroom		grape		spider monkey
	pickup		jelly fungus		elderberry		titi
	beach wagon		gill fungus		ffordshire bullterrier		indri
	fire engine		dead-man's-fingers		currant		howler monkey

14.3.3. GoogLeNet (Inception)

- *GoogLeNet (2015)*: introduce el *inception block* con caminos paralelos, cada uno con un filtro convolucional de tamaño distinto
- *Módulo inception*:

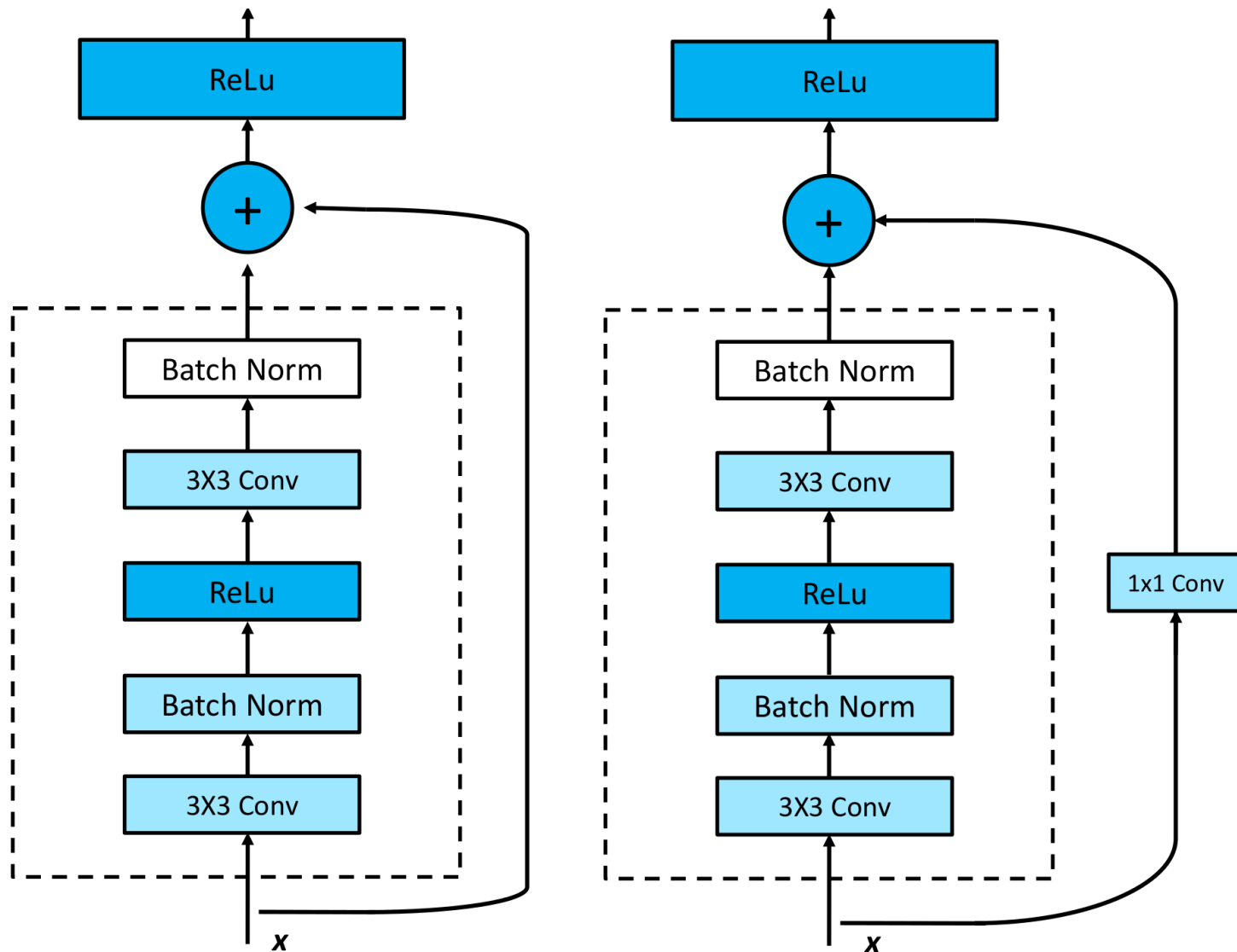


- ▷ 1×1 Conv reduce el número de canales
- ▷ Los caminos paralelos con convoluciones de tamaños distintos permiten escoger un tamaño de filtro diferente en cada capa

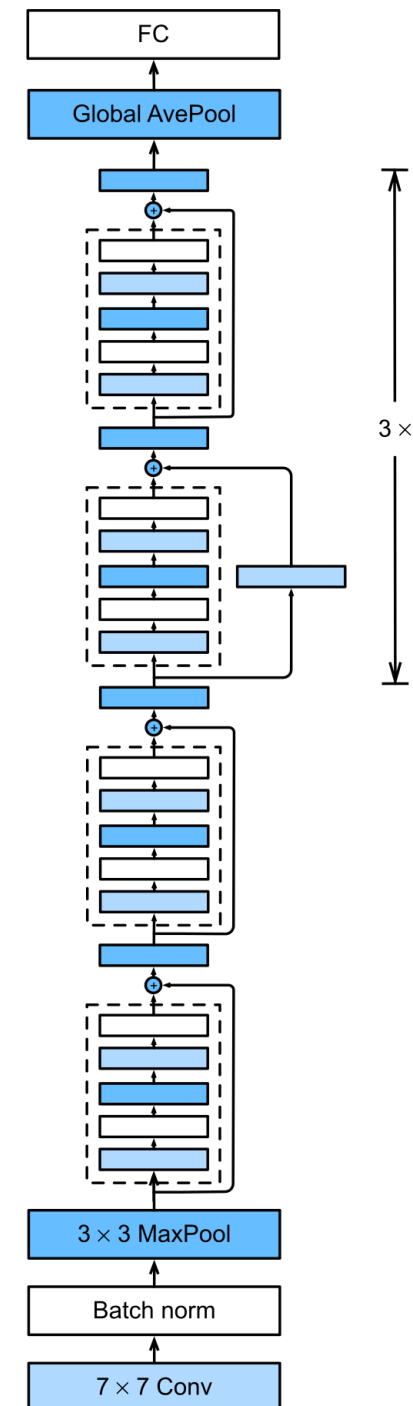


14.3.4. ResNet

- **Residual block:** reemplaza $x_{l+1} = \mathcal{F}_l(x_l)$ por $x_{l+1} = \varphi(x_l + \mathcal{F}_l(x_l))$ con $\mathcal{F}$ en versión estándar o (1x1Conv-)extendida

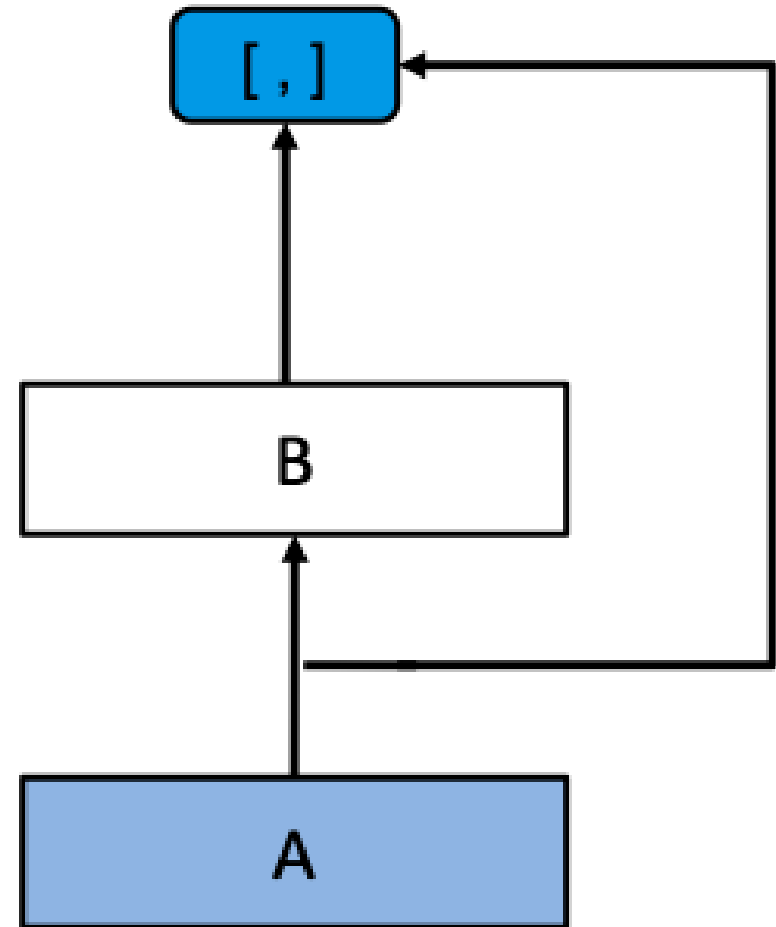
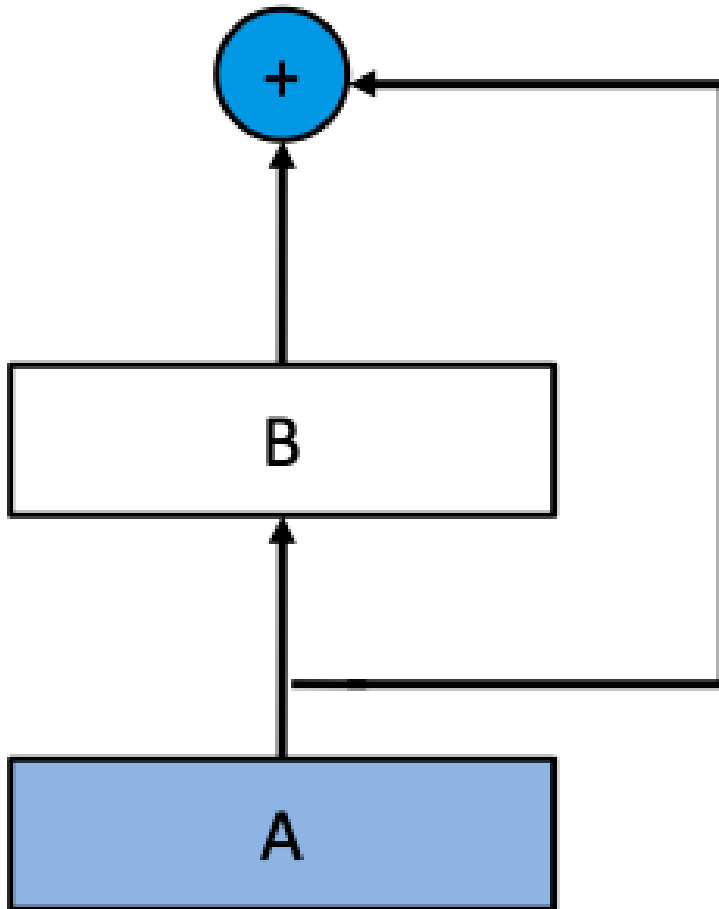


- **Resnet (2015):** apila muchos bloques residuales
- **Resnet-18 (dcha):** 18 capas entrenables: 16 correspondientes a 8 bloques residuales con 2 capas conv 3x3 cada uno, más la conv 7x7 (stride 2) inicial y la final completamente conectada
- **Preactivation resnet** o **PreResnet (2016):** facilita el entrenamiento de redes profundas reemplazando $x_{l+1} = \varphi(x_l + \mathcal{F}_l(x_l))$ por $x_{l+1} = x_l + \varphi(\mathcal{F}_l(x_l))$
- **Wide resnet (2016):** como alternativa al uso de modelos muy profundos, se propone el uso de modelos muy anchos, esto es, con muchos canales de características por capa

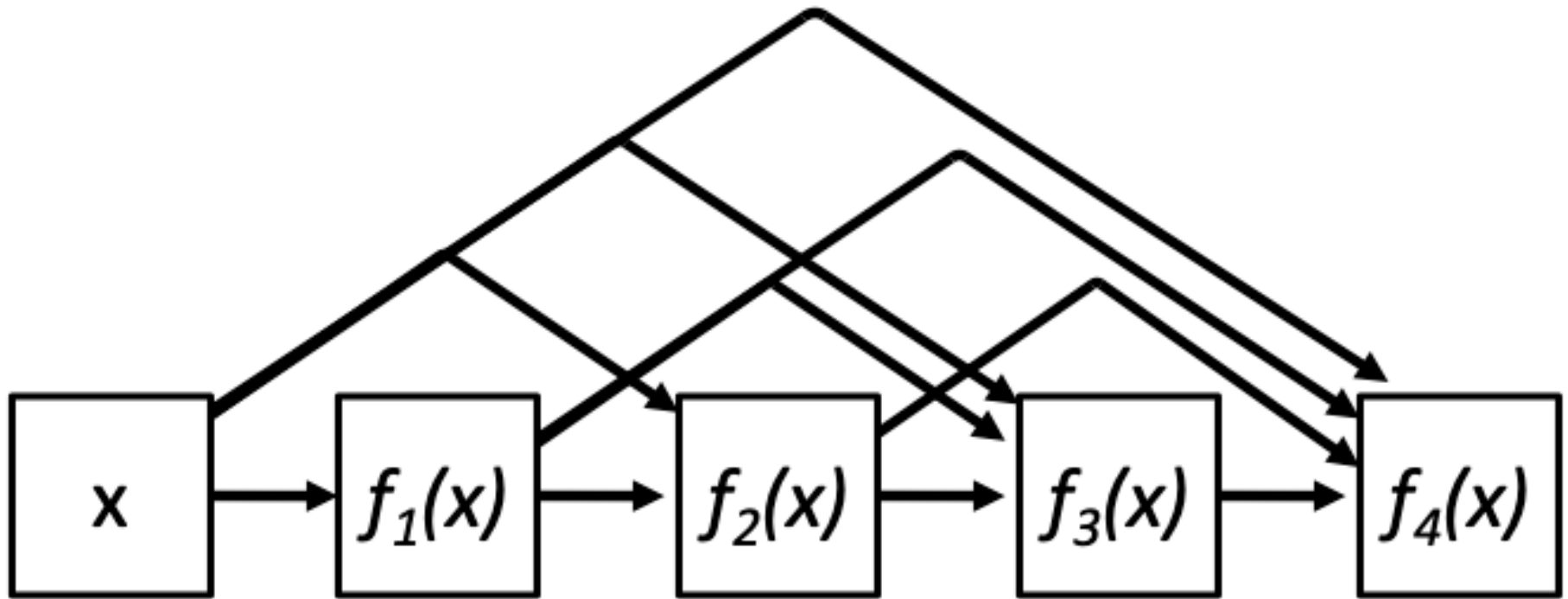


14.3.5. DenseNet

- *Bloque residual con concatenación* de salida y entrada (en lugar de suma)



- **DenseNet (2017):** apila bloques residuales con concatenación



- Elevado número de parámetros: reducible mediante adición de capas intermedias 1x1Conv y pooling (stride 2)

14.3.6. Búsqueda de arquitecturas neuronales

- ▶ *Auto-ML* o, en redes, *neural architecture search (NAS)*: exploración de modelos mediante optimización caja-negra
- ▶ *Optimización multi-objetivo*: precisión, tamaño del modelo, velocidad en aprendizaje e inferencia
- ▶ Área actualmente muy activa:

<https://github.com/topics/automl?o=desc&s=updated>

14.4. Otras formas de convolución

- Para segmentación y generación de imágenes

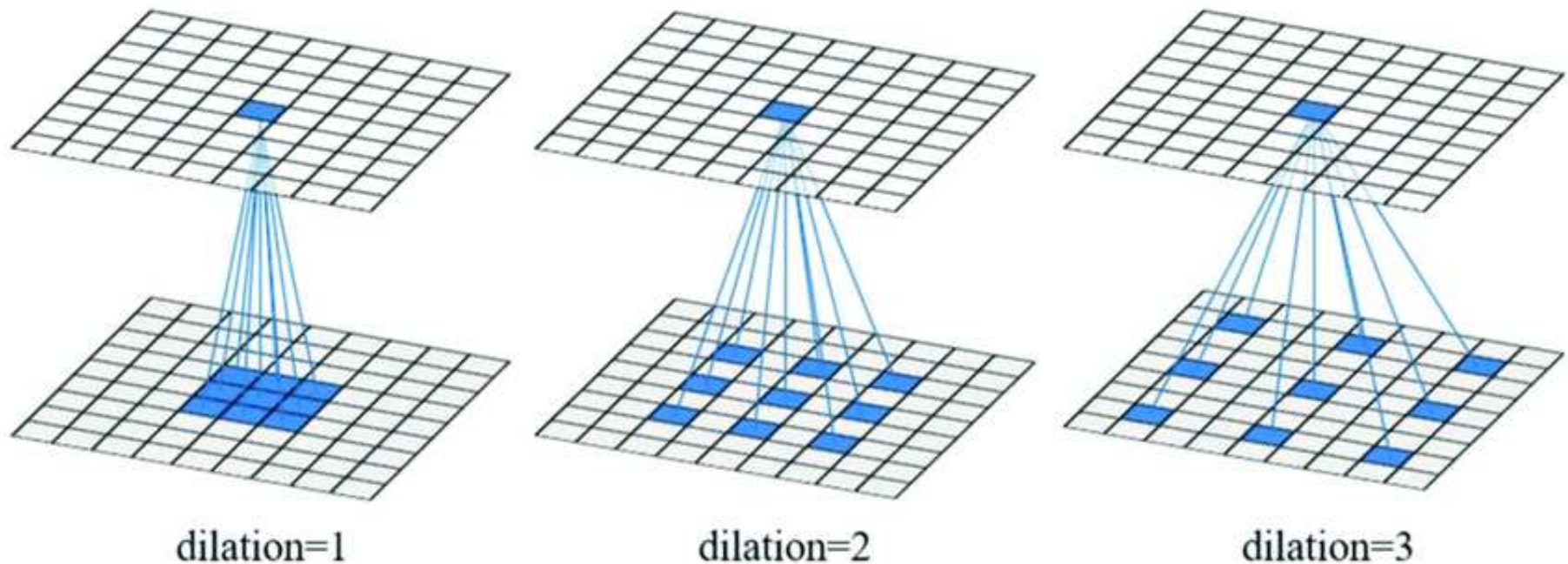
14.4.1. Convolución dilatada

- *Convolución dilatada, convolución con agujeros o à trous algorithm*: dado un *factor o ratio de dilatación* r , toma cada r -ésimo elemento de la entrada
- Ejemplo 1d: la convolución con un filtro w con un ratio $r = 2$ equivale a una convolución convencional con filtro $\tilde{w} = [w_1, 0, w_2, 0, w_3]$, donde los ceros se insertan para expandir el campo receptivo (de ahí el término “convolución con agujeros”)

► **Convolucióndilatada 2d:** con ratio r en altura y amplitud

$$z_{i,j,d} = b_d + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{i+ru,j+rv,c} w_{u,v,c,d} \quad (22)$$

► Ejemplo:



14.4.2. Convolución transpuesta

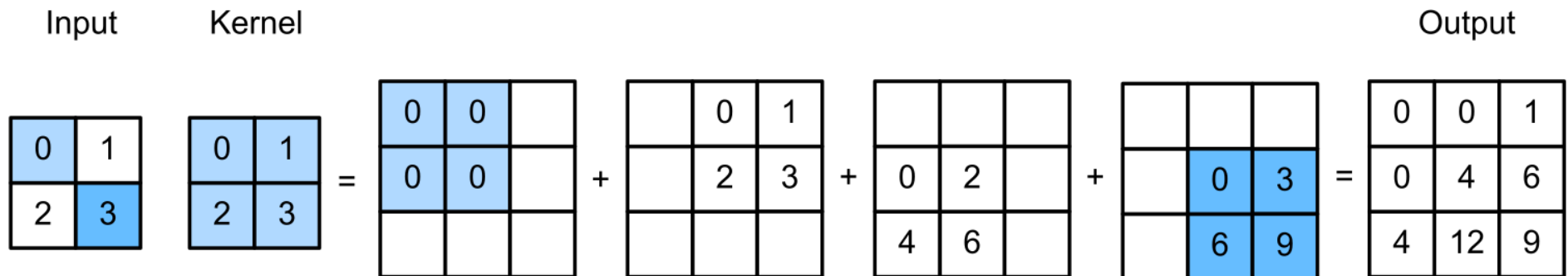
- Convolución convencional: reduce una entrada grande X a una salida pequeña Y mediante combinación ponderada de los píxeles de entrada y el kernel convolucional K

```
1 def conv(X, K):
2     h, w = K.shape
3     Y = zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
4     for i in range(Y.shape[0]):
5         for j in range(Y.shape[1]):
6             Y[i, j] = (X[i:i + h, j:j + w] * K).sum()
7     return Y
```

- **Convolución transpuesta:** produce lo contrario, esto es, una salida grande a partir de una entrada pequeña

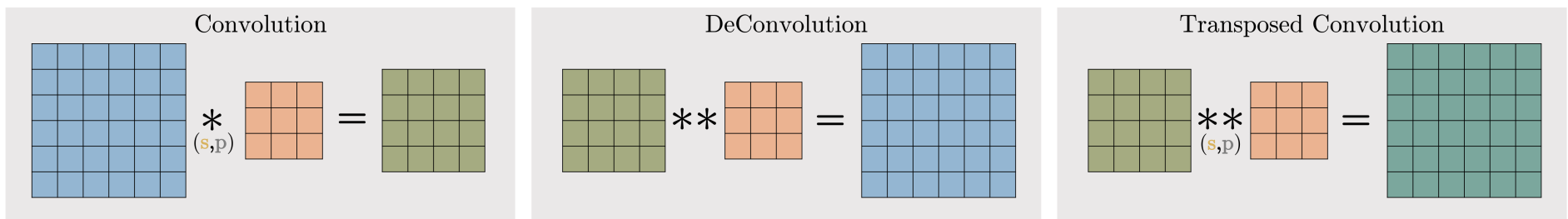
```
1 def trans_conv(X, K):
2     h, w = K.shape
3     Y = zeros((X.shape[0] + h - 1, X.shape[1] + w - 1))
4     for i in range(X.shape[0]):
5         for j in range(X.shape[1]):
6             Y[i:i + h, j:j + w] += X[i, j] * K
7     return Y
```

- *Ejemplo:* si (h, w) es el tamaño del kernel, la convolución transpuesta equivale a rellenar la imagen de entrada con $(h - 1, w - 1)$ ceros (por debajo a la derecha), colocar una copia ponderada del kernel en cada una de las posiciones de entrada (con peso igual al valor del píxel) y sumar



- Si W es la matriz derivada del kernel K , $Y = \text{trans_conv}(X, K)$ equivale a $Y = \text{reshape}(W^t \text{vec}(X))$ (de ahí “transpuesta”)

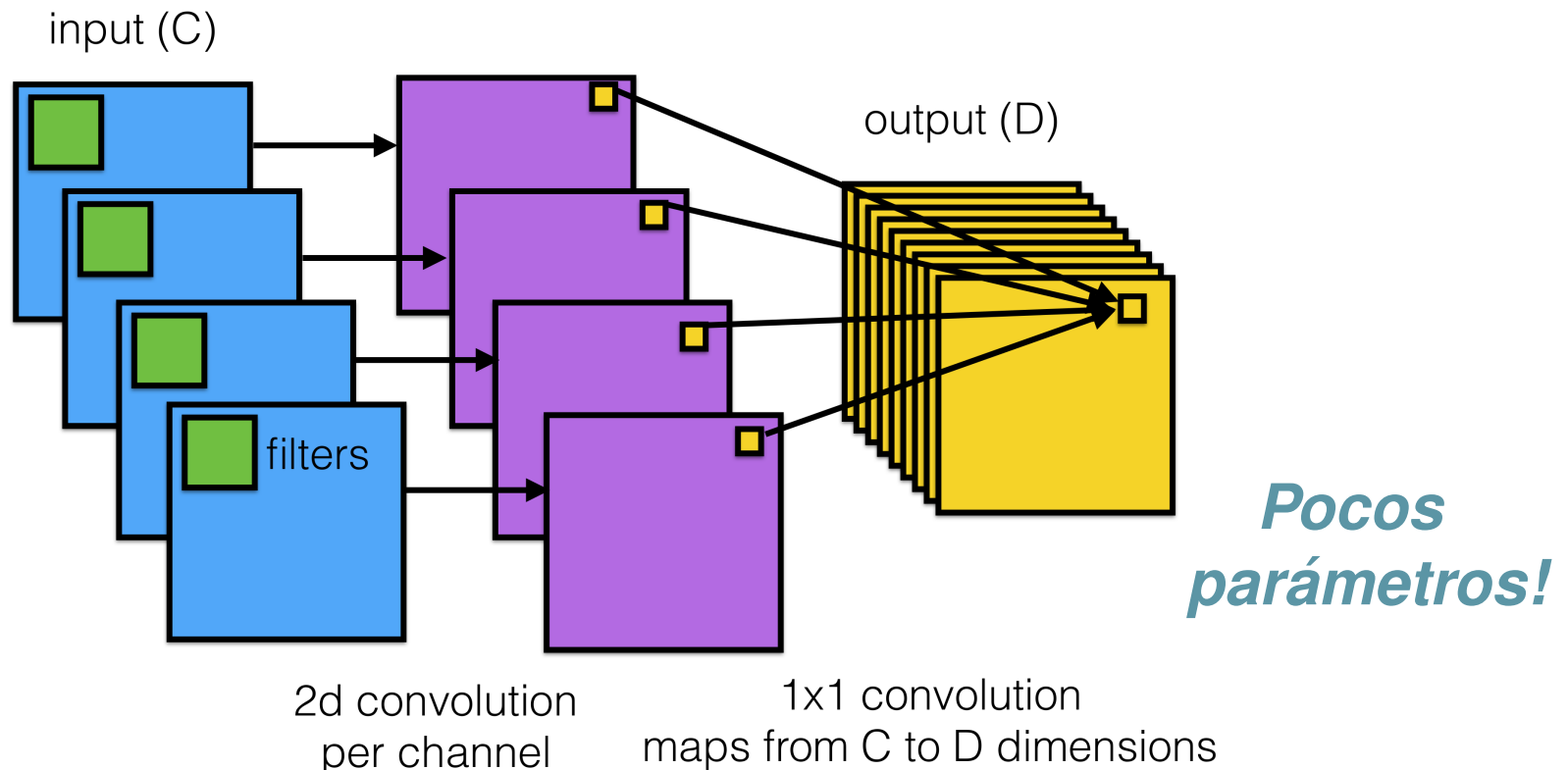
- **Deconvolución:** inversa de la convolución; distinta de transpuesta



14.4.3. Convolució separable en profunditat

- **Depthwise separable convolution:** convoluciona cada canal de entrada con su correspondiente filtro w y luego transforma los C canales resultantes en D mediante convolució 1x1 w'

$$z_{i,j,d} = b_d + w'_{c,d} \sum_{c=0}^{C-1} \left(\sum_{u=0}^{H-1} \sum_{v=0}^{W-1} x_{i+u,j+v,c} w_{u,v} \right) \quad (23)$$



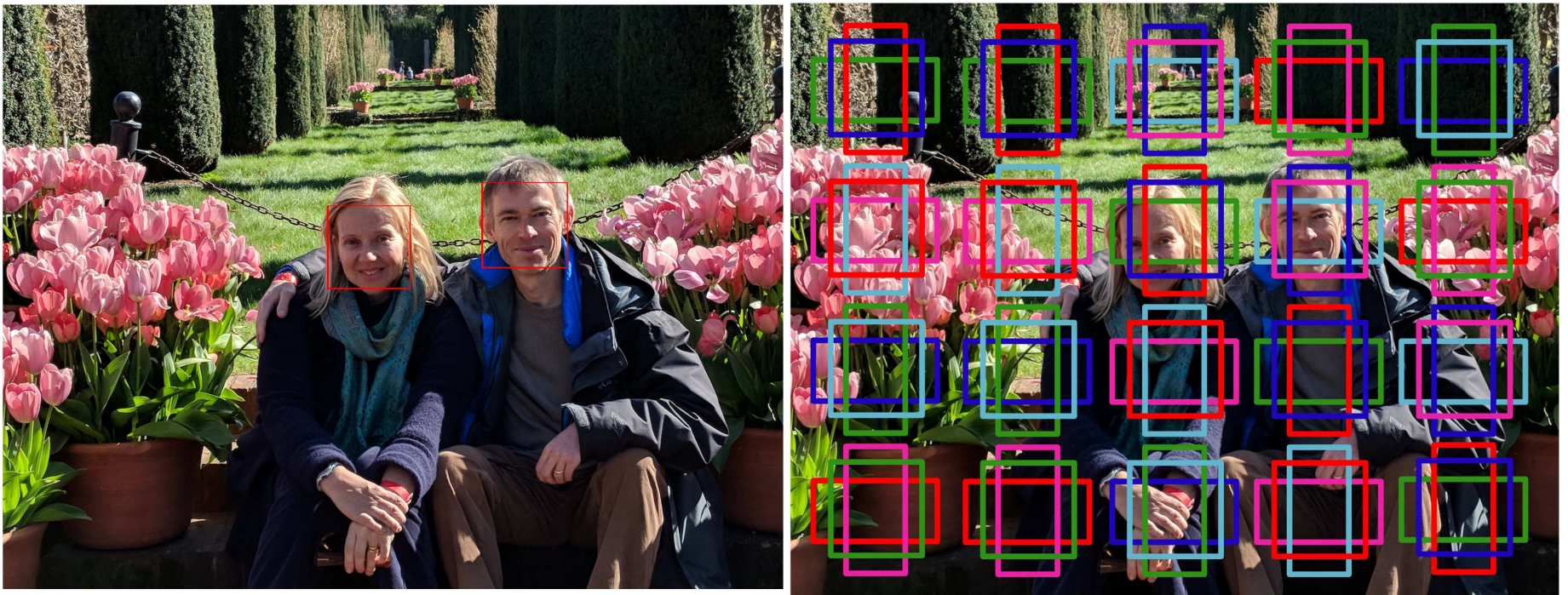
14.5. CNNs para otras tareas de visión

14.5.1. Etiquetado de imágenes

- *Image tagging*: clasificación multi-etiqueta de imágenes según los objetos presentes

14.5.2. Detección de objetos

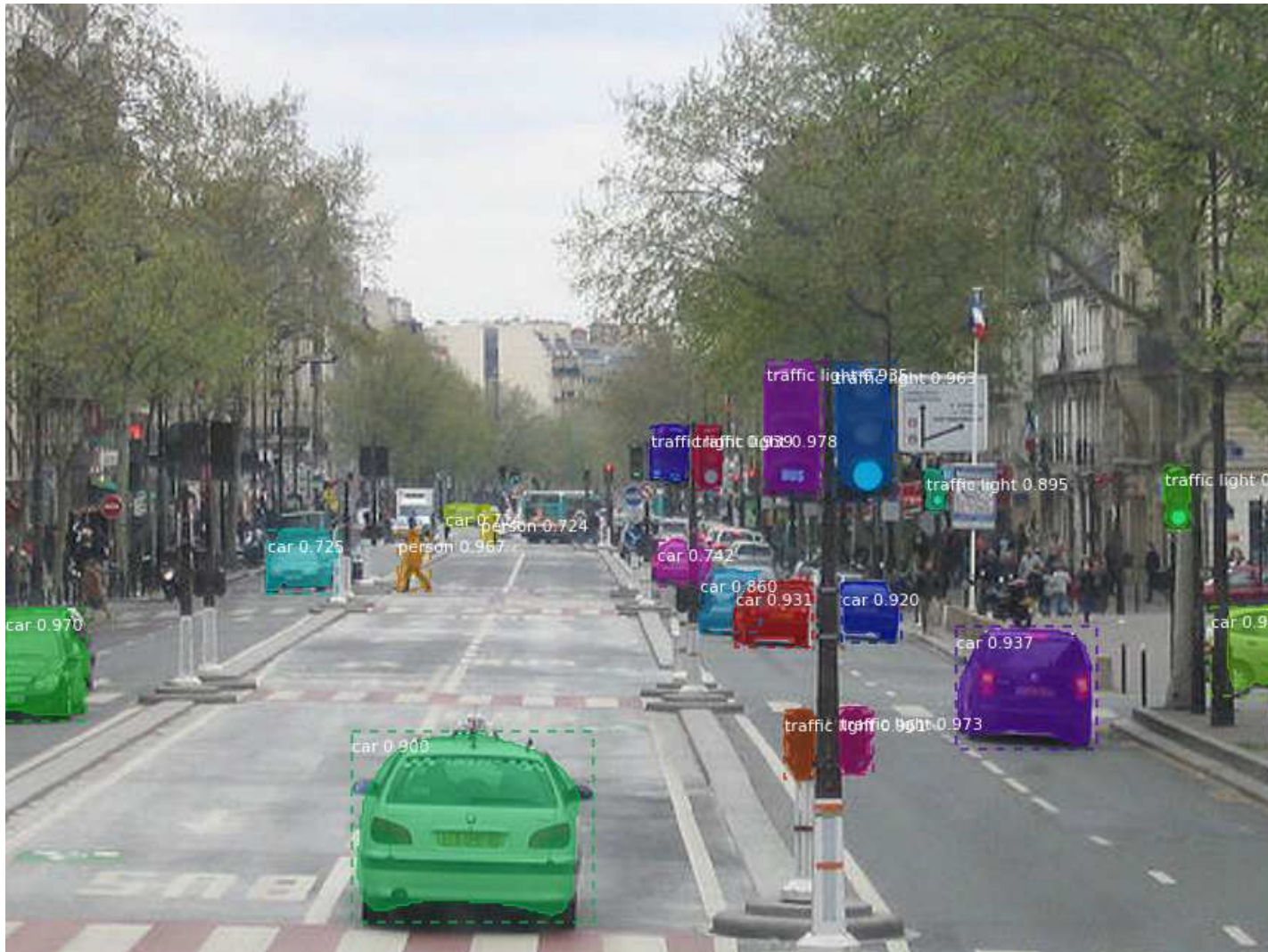
- **Object detection:** produce cajas de mínima inclusión donde se hallan los objetos de interés, así como sus etiquetas de clase
- Ejemplo: **detección de caras** mediante **cajas ancla**



Por cada posición de caja (i, j) , predecimos una probabilidad de presencia de objeto, categoría de objeto y esquinas de la caja

14.5.3. Segmentación de instancias

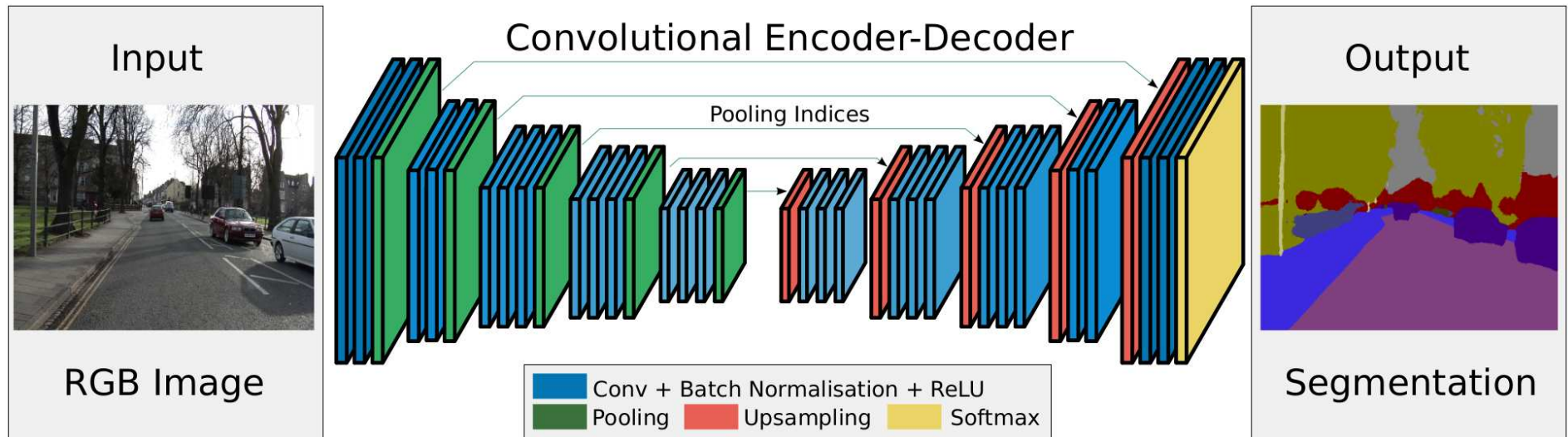
- *Instance segmentation*: produce una *máscara de forma 2d* y una etiqueta de clase por cada instancia de objeto



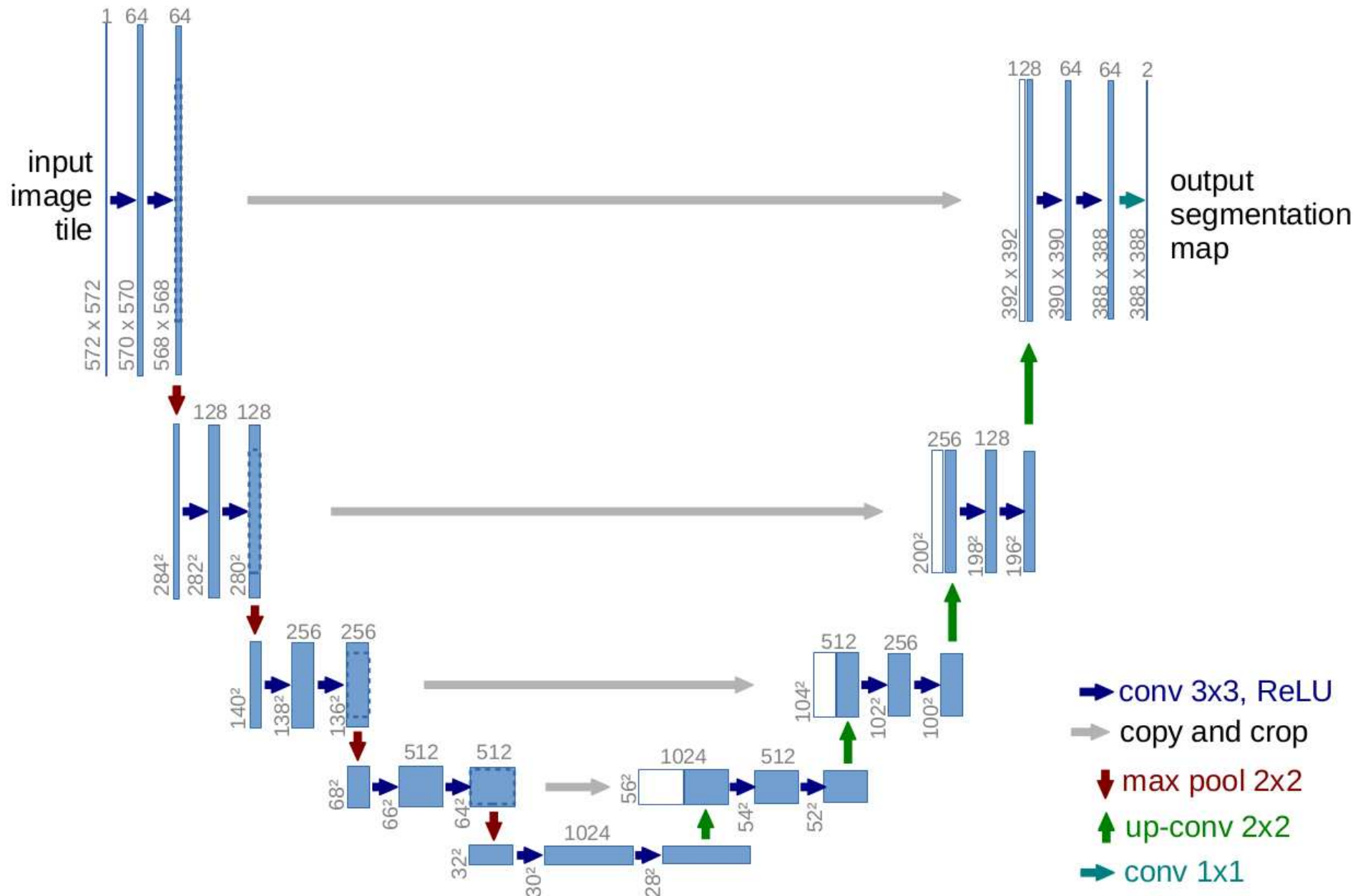
14.5.4. Segmentación semántica

- ▶ ***Semantic segmentation:*** produce una etiqueta de clase por cada píxel (sin diferenciar objetos como segmentación de instancias)
- ▶ ***Arquitectura encoder-decoder:***
 - ▷ ***Encoder:*** aplica convoluciones (dilatadas) estándar para transformar la entrada en un cuello de botella 2d que captura propiedades de alto nivel a baja resolución espacial
 - ▷ ***Decoder:*** devuelve el cuello de botella 2d a una imagen de salida de tamaño completo mediante convoluciones transpuestas
 - ▷ Se añaden conexiones de salto de las capas de entrada a las de salida ya que el cuello de botella pierde información

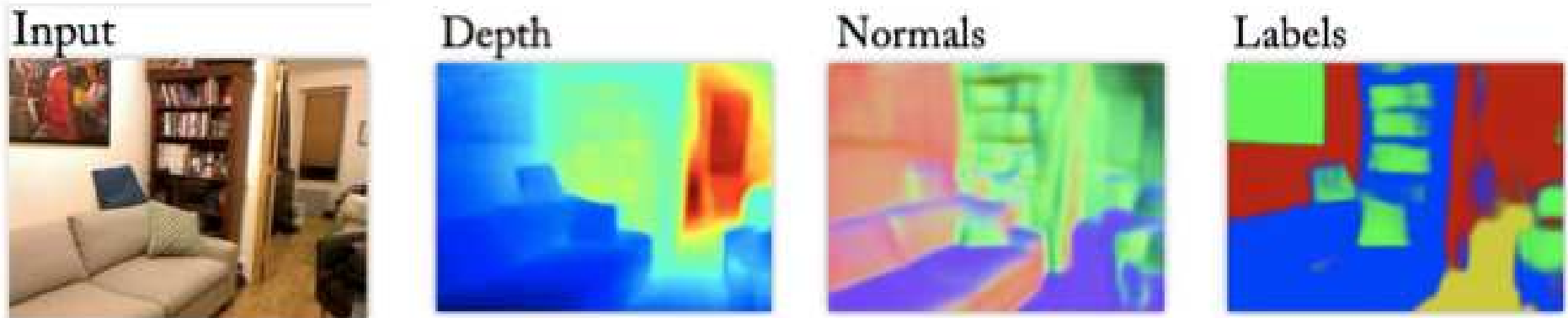
► *Ejemplo:* CNN encoder-decoder para segmentación semántica



► **U-net:** arquitectura encoder-decoder representada en forma de U



- ▶ U-net se aplica en tareas de *dense prediction* o *image-to-image* como *depth prediction* y *surface normal prediction*



- ▶ *Panoptic segmentation*: combina segmentación semántica y de instancias de manera coherente

14.5.5. Estimación de pose humana

- *Human pose estimation*: detección de personas y predicción de sus formas 2d, bien con máscaras o mediante localización de un conjunto fijo de puntos esqueléticos

